KUBERNETES STATEFUL APPS



learnk8s

Table of contents

1. Managing state	**5**
Storing configuration	7
Storing secrets securely	9
Persisting changes with Volumes	13
The emptyDir Volume	17
Volume drivers in Kubernetes	20
Using Secrets and ConfigMaps as Volumes	22
Mounting local folders as Volumes	23
Using Volumes from a cloud provider	27
Abstracting volumes for portability with Persistent Volumes	32
Persistent Volume access modes	34
Using Persistent Volumes with Persistent Volume Claims	36
Using Persistent Volumes as Volumes in your Pods	37
Dynamically provisioning of Persistent Volumes	39
Using local volumes as an alternative for hostPath	42

Mounting custom Volume drivers 46

Designing a clustered PostgreSQL 48

Creating a clustered database using StatefulSets 54

How the StatefulSet controller works 62

Creating a controller for a clustered database 65

Database operators 70

Distributed storage with operators 72

Using OpenEBS to launch a clustered database 75

2. Deploying a stateless MySQL 80

Prerequisites 81

Creating the Deployment 81

Storing credentials in Secrets 84

Verifying that you can connect to MySQL 86

3. Persisting changes 88

Creating a Persistent Volume 90

Claiming a Persistent Volume 91

Using volumes in Deployments 92

Testing the persistence 94

4. Dynamic provisioning 96

Creating volumes with a Storage Class 98

Testing the Deployment 101

5. Lab 103

Extracting configs 104

Chapter 1

Managing state

Imagine having an application that connects to a database.

If the database already exists, you can use its connection string in your app.

Using the URL and hardcoding it inside the container and your Pod isn't a brilliant idea, though.

A better option is to extract it as a parameter and provide it at run time — when the Pod starts.

Ideally, you want to provide a distinct URL for development, preproduction and production environments.

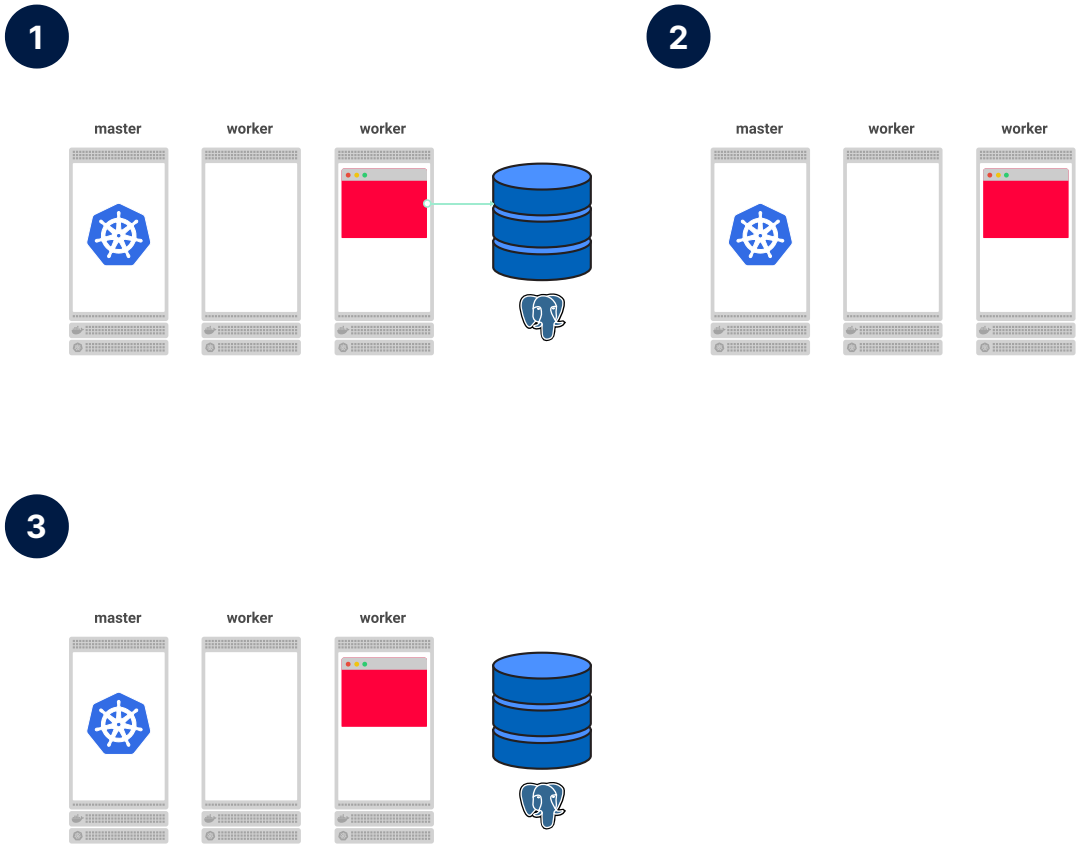


Fig. 1 Imagine having an application that connects to a database.

Fig. 2 You might have an existing database already that lives outside the cluster.

Fig. 3 You could use the connection string to connect to the database from within the cluster.

Connecting to a URL isn't probably enough either.
The database is protected using username and password.
So you should pass those alongside the connection string when you want your application to connect to it.
Username and password are sensitive information, though.
While you could share the connection string with the rest of the world, you don't want to do the same with username and password.
Kubernetes has two objects to store configuration and sensitive data: ConfigMaps and Secrets.

Storing configuration

ConfigMaps are objects that contains key-value pairs.

You can use ConfigMaps to store configurations that change across environments such as a feature flag that is enabled in production, but not in development or preproduction.

ConfigMaps are the perfect medium to store the address to connect to your database.

configmap.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  database_url: postgresql://my-postgres:5432
```

Once you have decided which values you wish to store, you can use the ConfigMap in your Pod.

It's common practice to use the values stored in the ConfigMap as environment variables inside the Pod.

You could inject the URL of the database as a `DATABASE_URL` environment variable in the Pod.

pod.yaml

```yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
```

```
            name: my-config
            key: database_ur
```

Kubernetes is designed to be stateless.

So, where are the objects such as ConfigMaps and Pods stored?

If you guessed etcd, you're right.

All Kubernetes objects are stored inside etcd.

So if you had access to etcd, you could inspect etcd, find the ConfigMap and retrieve its values.

Kubernetes has a second object similar to ConfigMaps called Secret.

Storing secrets securely

Secrets look precisely like a ConfigMap, and they are a collection of key-value pairs.

One of the differences, though, is that Secrets have their values encoded using base64.

Secrets are designed to store sensitive information such as the username and password for a database.



secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  username: bGVhcm5rOHMK
  password: cHJlY2lvdSQK
```

You can use a Secret in your Pods in the same way as a ConfigMap.

In this case, you can inject two environment variables for username and password from a Secret.

pod.yaml

```yaml
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      env:
        - name: DATABASE_USERNAME
          valueFrom:
            secretKeyRef:
              name: my-SECRET
              key: username
```

Base64 is a binary-to-text encoding scheme.

It's not designed to encrypt secrets.

If someone gets hold of your etcd database, they could grab your Secrets, decode the base64 values and run away with your passwords.

And what's the point of having a Secret when you could have a ConfigMap where all the values are manually encoded to base64?

Base64 is not meant to encode or hide the values inside a Secret.

The encoding is used to facilitate storing files such as certificates as strings.

You could do the same with ConfigMaps too.

But Secrets come with extra features that make them different *enough* from ConfigMaps.

You can define an encryption mechanism at rest for Secrets.

So before the Secret is stored in etcd and after the values are submitted to the API, a provider of choice such as a Key Management Service (KMS) encrypts the Secret.

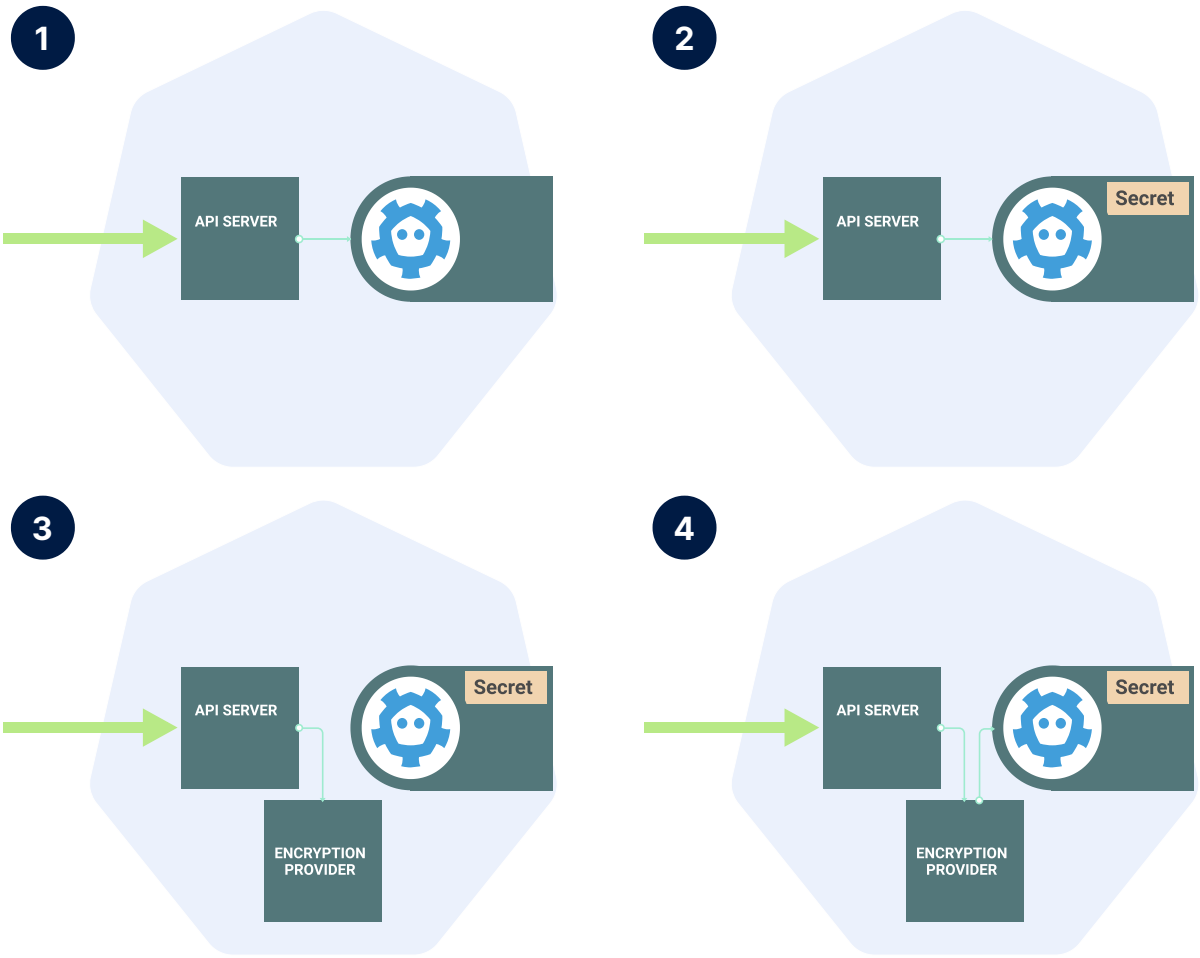


Fig. 1 When you create a Secret with `kubectl apply -f secret.yaml`, Kubernetes stores it in etcd.

Fig. 2 When you create a Secret with `kubectl apply -f secret.yaml`, Kubernetes stores it in etcd.

Fig. 3 The Secrets are stored in clear in etcd unless you define an encryption provider.

Fig. 4 When you define the provider, before the Secret is stored in etcd and after the values are submitted to the API, the Secrets are encrypted.

Please note that by default, the encryption is turned off.
All your Secrets are stored in clear in etcd.

> You can read more about enabling encryption in your Secrets on the official documentation.

With Secret and ConfigMap you can store all the details that you need to connect your Pods to a database securely.
But what if you want to host the database in Kubernetes rather than connecting to an external one?
You know how to store the details in ConfigMaps and Secrets.
However, what you need is a way to persist the data on disk.

Persisting changes with Volumes

You might have noticed that every time a container is restarted inside a Pod, the data on the filesystem is preserved and ready to be used after the restart.
In other words, everything works as usual when a container is restarted.
However, when a Pod is removed, the data is lost too.
That's not great news if you wish to have your database

running as a Pod.

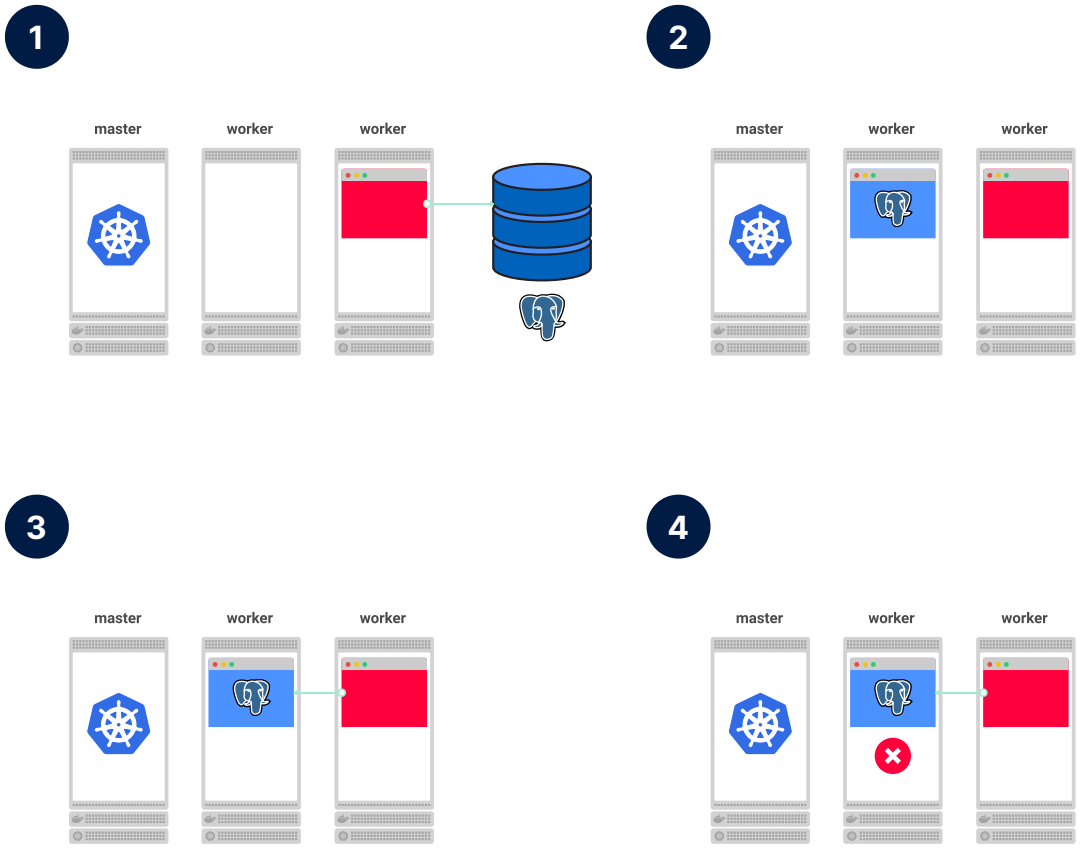


Fig. 1 When you have a database external to the cluster, you can use ConfigMap and Secrets to store the details to connect to it.

Fig. 2 When the database is hosted as Pod and deployed as part of your application, you can use a Service to talk to the database (please note that you still need a Secret for username and password).

Fig. 3 Having the database deployed as a Pod in the cluster makes it more portable as you can deploy everything at once without having to provision infrastructure somewhere else.

Fig. 4 Unfortunately, if you leave everything as-is, when the database Pod is deleted, so is its data.

There should be a better way to persist the changes locally and in the cloud.

And there is.

Kubernetes has a concept of Volume where you can persist changes to a storage of choice.

A Volume is mounted into the Pod and unmounted when the Pod is deleted.

In other words, the lifecycle of a Volume is the same as the Pod: you can't have a volume mounted longer than the Pod it is attached to.

Also, Volumes are attached to Pod and not containers.

If a container restarts, the Volume stays mounted.

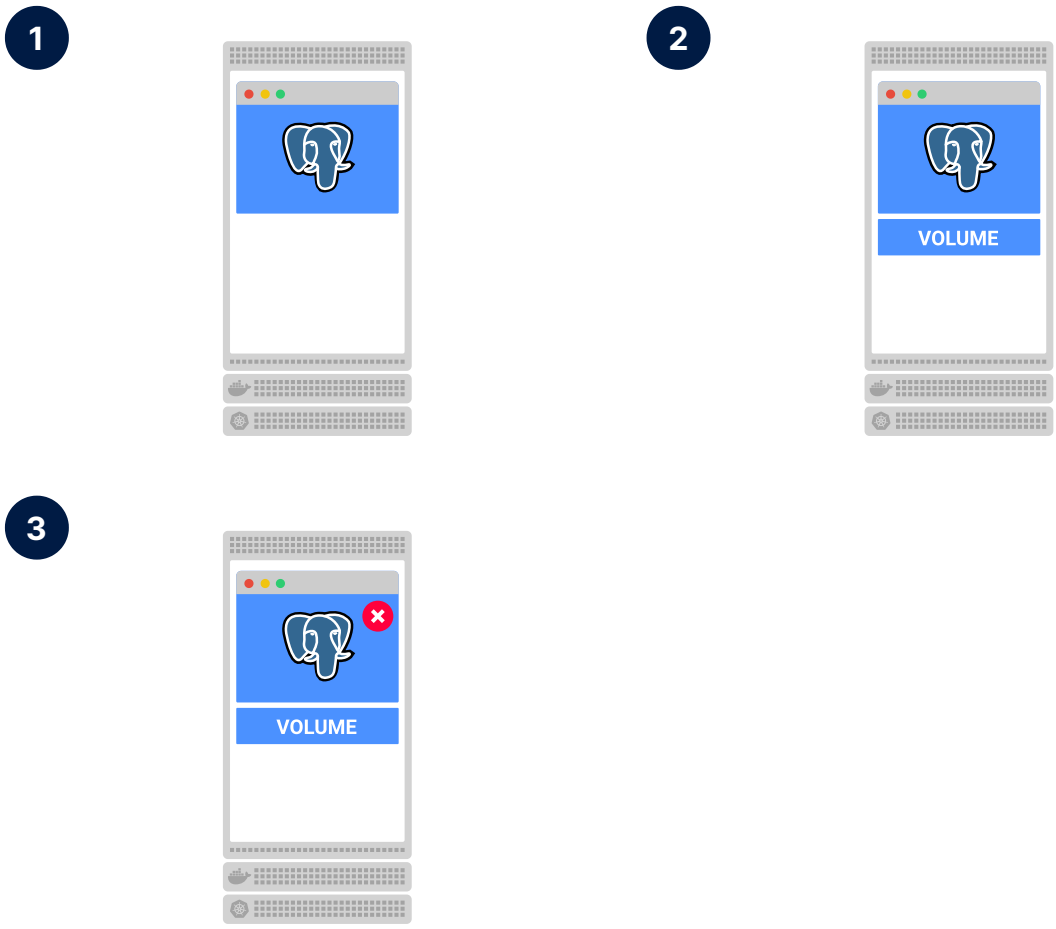


Fig. 1 When you wish to persist data into a Pod, you can use a Volume.

Fig. 2 When you wish to persist data into a Pod, you can use a Volume.

Fig. 3 Even if one of the containers inside the Pod restarts, the Volume remains mounted.

The containers inside a Pod are already using a Volume because you can write to the filesystem from one container

and see the changes in another.
They're using an emptyDir Volume.

The emptyDir Volume

The emptyDir volume is first created when a Pod is assigned to a Node and exists as long as that Pod exists.
The volume is empty when it is mounted, and it is deleted when unmounted.
And it is created automatically with every Pod.
However, you could create it manually with:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - image: k8s.gcr.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir: {}
```

When you create a Volume, you can mount it inside your Pod and assign a folder to it.

In this case, the Pod is mounted as `/cache` inside the container.

pod.yaml

```yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - image: k8s.gcr.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir: {}
```

Please note that the mount path is independent and specified to the container.

If you have multiple containers in your Pod, you could mount the same Volume more than once on different paths.

EmptyDir Volume is convenient so that all containers on the same Pod can see and share a common filesystem.

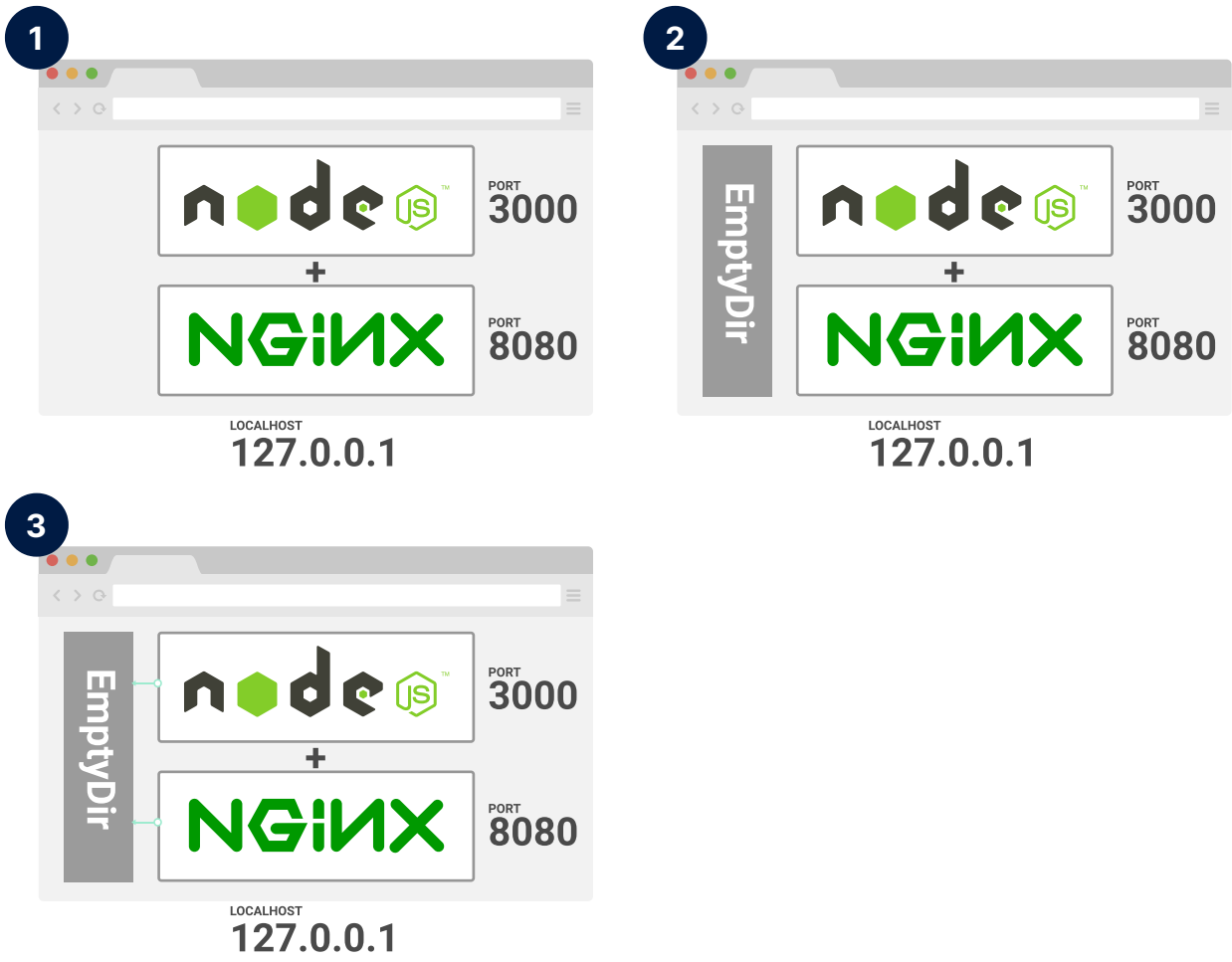


Fig. 1 A Pod can hold multiple containers. All of the containers share the same network namespace.

Fig. 2 Containers inside a Pod also share Volumes. If you don't define a Volume, an emptyDir volume (a local folder in the node) is automatically mounted in the Pod and share by containers.

Fig. 3 Containers can read and write to the shared filesystem and files created by one container can be used by others.

emptyDir isn't the Volume you can use, but it's the default. Kubernetes comes with several Volumes drivers.

Volume drivers in Kubernetes

Here's a list of Volumes you could use:

- awsElasticBlockStore
- azureDisk
- azureFile
- cephfs
- configMap
- **csi**
- downwardAPI
- **emptyDir**
- fc (fibre channel)
- **flexVolume**
- flocker
- gcePersistentDisk
- glusterfs
- **hostPath**
- iscsi
- **local**

- nfs
- **persistentVolumeClaim**
- projected
- portworxVolume
- quobyte
- rbd
- scaleIO
- **secret**
- storageos
- vsphereVolume

Exploring each Volume isn't in the scope of this section. Instead, you will focus on a few important Volumes and drivers:

- emptyDir
- configMap and secret
- hostPath
- persistentVolumeClaim
- Local
- flexVolumes and CSI

You already learned about the EmptyDir volume and how it's the default Volume in a Pod.

You also just learned about ConfigMaps and Secrets and how you can use their values to inject environment variables in your Pods.

There's a second way to use ConfigMaps and Secrets.
You can mount them in your Pods as Volumes.

Using Secrets and ConfigMaps as Volumes

When you mount a Secret or ConfigMap as a volume, the values are written to disk as files.
You could have a ConfigMap with the content of an `application.yml` — the externalised configuration file for Spring boot.

configmap.yaml

```yaml
kind: ConfigMap
apiVersion: v1
metadata:
  name: hello-world-config
data:
  application.properties: |
    greeting=Hello
```

You could mount the content as a file from the ConfigMap into the Pod with:

pod.yaml

```
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: spring-boot
    volumeMounts:
    - name: config
      mountPath: /config
  volumes:
  - name: config
    configMap:
      name: hello-world-config
```

In this example, you could expect the file `/config/application.properties` to contain the values `greeting=Hello`.

You could use a similar strategy to mount certificates from Secrets inside an Ingress Pod.

Mounting local folders as Volumes

Another noteworthy Volume is hostPath.

With a hostPath Volume, your Pods can read files and directories from the Node they are currently on.

Files written to disk in the Nodes are not deleted when the Pod is deleted.

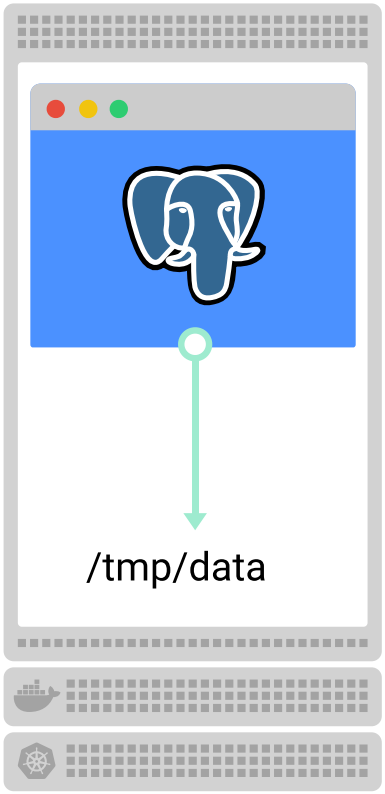


Fig. A Pod can store write to the Node's filesystem using a hostPath Volume

The hostPath is the most straightforward Volume to store data permanently.
Since you could mount any file or directory on the Node, hostPath is always useful when you want to:

- mount and control the GPIO on your Raspberry PI, and you need access to `/dev/gpiomem`

- Mount the socket for the Docker daemon, or have access to the Docker logs inside a node. A useful way to read and ship the logs to a central location
- Mount specific hardware such as GPU resources

Using hostPath sounds great.
But you should be aware that:

- The Pod has to have elevated privileges to read and write to the Node
- The data doesn't follow your Pod.

And paying attention to the last point is critical.

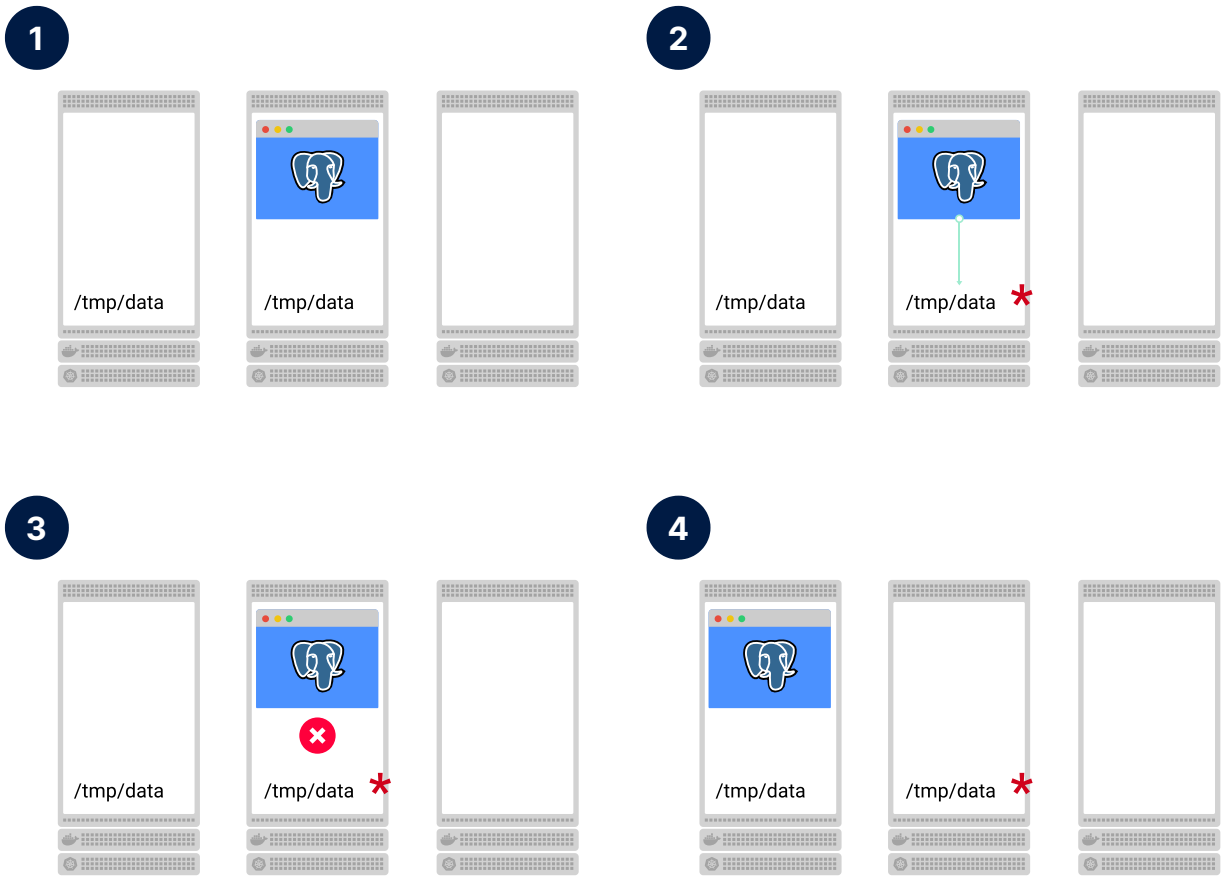


Fig. 1 Imagine you have three nodes and decide to schedule a Pod with a hostPath. The scheduler picks one node and assigns the Pod.

Fig. 2 The Pod runs and starts writing to disk.

Fig. 3 The Pod is unfortunately deleted soon after it wrote some data to disk.

Fig. 4 The replication controller respawns the Pod, but this time the scheduler assigns it to another Node. The Pod doesn't find the same data as before.

When the Pod is rescheduled to a different Node, it accesses that folder for the first time, so the Pod has to start from zero.

In other words, the data is not replicated and the state as left as is.

Also when a Node is removed perhaps because of the cluster shrinking as a consequence of autoscaling, the data is lost too.

While you can use hostPath, you should pay attention and work around its drawbacks particularly around data retention.

Using Volumes from a cloud provider

A better strategy would be to use a Volume backed by a cloud provider such as awsElasticBlockStore or azureFile.

Assuming the volumes are created ahead of time, you can attach them to the Pod and start using them.

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
```

```
    name: test-ebs
spec:
  containers:
  - image: k8s.gcr.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /test-ebs
      name: test-volume
  volumes:
  - name: test-volume
    # This AWS EBS volume must already exist.
    awsElasticBlockStore:
      volumeID: <volume-id>
      fsType: ext4
```

If the Node is deleted, the Volume is unmounted.

When the Pod is scheduled on a different Node, the volume can be moved and remounted to the Pod while still retaining the original data.

Overall, using a Volume provided by a cloud provider leads to a much better experience and usability.

It comes at a cost, though: **the Volume type is hardcoded in the Pod definition**.

If one day you decide to move from Amazon Web Services to Azure, you will have to rewrite all your Pod definitions — and that includes Deployments, DaemonSets, StatefulSets and all other resources that reference the volume.

Even if you are in love with Amazon Web Services and have no plans to move you still face the same challenges when Amazon Web Services decides to upgrade their volumes to new and better storage.

Lastly, if you plan to share your work as Open Source Software, you don't want to dictate what Volume your users should use.

In those cases, it'd be better to use an abstraction where the user can decide how to plug-in their storage of choice.

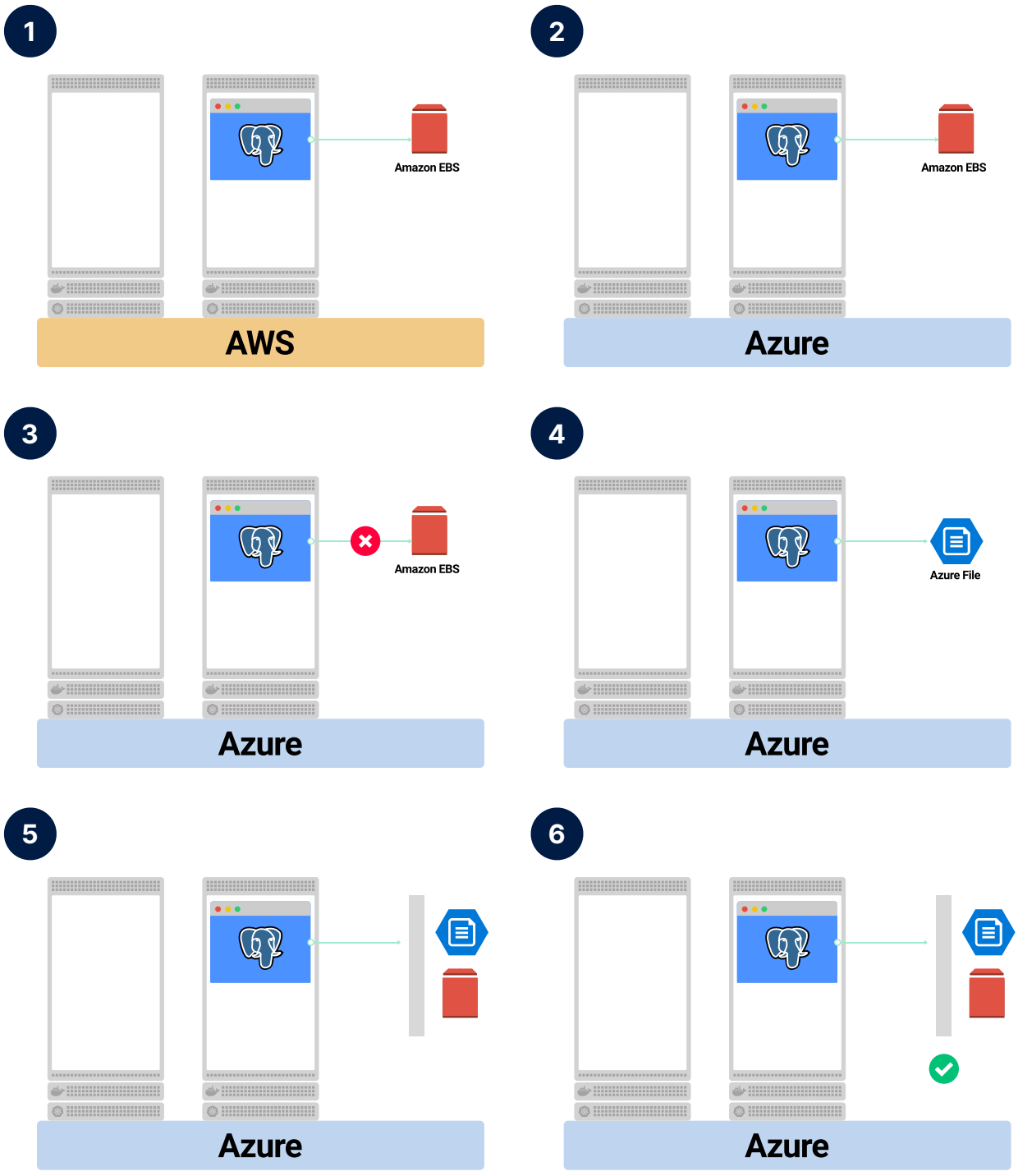


Fig. 1 You could use a Volume from a cloud provider such as awsElasticBlockStore. The Volume is mounted and moved to the right Pod and Node when necessary, and the data is preserved even if the Pod is lost.

Fig. 2 The definition for the awsElasticBlockStore Volume is hardcoded in the Pod. What happens when you wish to run your Deployments in Azure as-is?

Fig. 3 Azure offers azureFile and azureDisk, but it doesn't offer an awsElasticBlockStore. None of your Pods will run.

Fig. 4 You could rewrite all your Pod definitions (as well as Deployments, ReplicaSets, DaemonSets, StatefulSets, etc.) and replace the awsElasticBlockStore Volume for azureDisk, but that's a lot of work.

Fig. 5 Rather than being locked-in into a cloud Provider's Volume, it'd be better to create an abstraction to deal with the complexity of selecting the right volume.

Fig. 6 It shouldn't matter which cloud provider or infrastructure you use. You should be able to run Volumes regardless without rewriting your resource definitions.

In other words, you should be able to request storage, and whoever is running the cluster should be able to provide the storage you asked for.

But they should be in charge of selecting the appropriate Volume.

Persistent Volumes and Persistent Volume Claims are designed to solve that.

Abstracting volumes for portability with Persistent Volumes

Before you can use an external Volume, you need to map it and make the cluster aware of it.

A Persistent Volume is designed to be a placeholder for external storage that was created outside the cluster.

The external storage could be an Elastic Block Store (EBS) volume in Amazon, an Azure Disk in Azure or a storage device in your cloud provider of choice.

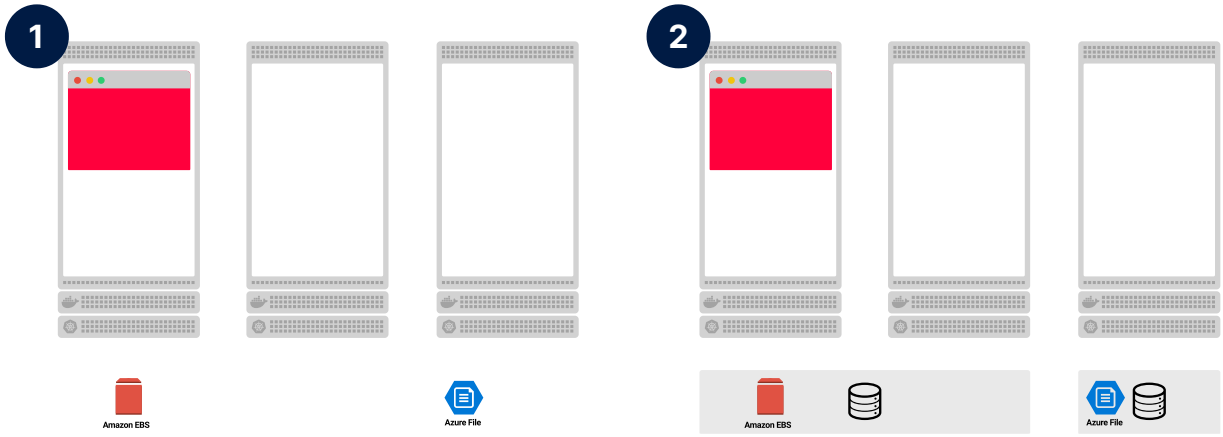


Fig. 1 You might have Volumes that are provisioned outside your cluster.

Fig. 2 You can map those Volumes native Kubernetes resources called Persistent Volumes.

Please note that those disks such as Elastic Block Store and Azurefile are not part of the cluster and are provisioned manually.
For each storage you wish to use you should create an equivalent Persistent Volume in Kubernetes defining the specs of that volume.

pv.yaml

```yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: task-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteOnce
  awsElasticBlockStore:
    volumeID: vol-867g5kii
    fsType: ext4
```

Persistent Volumes are not related to Pods at all and can outlive all of your Pods in the cluster.

Two metrics are essential when defining and mapping volumes: **capacity and access mode**.

The capacity is how much storage capacity is available in the Volume.

The access mode describes how the Volume can be accessed.

Is it read-only? Read and write?

Can multiple consumers just read?

Or maybe read and write?

Persistent Volume access modes

The access mode isn't related to Pods though but to Nodes.

If you think that storage is equivalent to attaching a physical disk to a Node, it makes sense that access mode is linked to Nodes and not Pods.

Can a disk connected to a Node be available to another Node?

It depends by the disk.

Perhaps a Volume can be writable from a single Node and readable from all the others.

All the Pods scheduled on that particular Node can read and write to that Volume.

But all other Pods will be restricted to reading only.

There're three access modes:

- ReadWriteOnce (RWO) – the Volume can be mounted as read-write by a single node
- ReadOnlyMany (ROX) – the Volume can be mounted read-only by many nodes
- ReadWriteMany (RWX) – the Volume can be mounted as read-write by many nodes

Please note that a Volume can only be mounted using one access mode at a time, even if it supports many.

As an example consider an NFS Volume.

NFS volumes can be used as ReadWriteOnce, ReadOnlyMany and ReadWriteMany.

However, when you create the Persistent Volume (the mapping between the real storage and the Kubernetes resource), you might decide to map the volume as read-only.

You won't be able to use the volume read-write-many with that Persistent Volume.

Unless you redefine it.

When you wish to use a Persistent Volume, you can't directly mount it in the Pod.

That would defeat the purpose of abstracting the underlying storage and would create the same tight coupling we discussed earlier.

Before you can use a Persistent Volume, you need to claim it.

Using Persistent Volumes with Persistent Volume Claims

Persistent Volume Claims are Kubernetes objects designed to request for a storage resource.

Imagine you are developing an app that let the user upload a video, apply filters and download it.

Before you create the app, you should request for a volume with 20GiB of free space, so that you can store the videos somewhere.

If no Persistent Volume can satisfy your request, the Persistent Volume Claim stays pending until there's a Persistent Volume with the right specs.

If there'a Persistent Volume available with at least 20GiB of space, the Persistent Volume Claim binds to the Persistent Volume, and no one else is allowed to use it.

There's a 1-to-1 relationship between Persistent Volumes and Persistent Volume Claims.

You can't have more than one Persistent Volume Claim claiming the same Persistent Volume.

Persistent Volume Claims look like this:



pvc.yaml

```
kind: PersistentVolumeClaim
```

```
apiVersion: v1
metadata:
  name: task-pv
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
```

You claimed the Volume, but you haven't used in your Pod yet.

Using Persistent Volumes as Volumes in your Pods

You can use Persistent Volumes as Volumes in your Pods.

Depending on the access mode on your Persistent Volume Claim you can have multiple Pods sharing the same Persistent Volume Claim and underlying Persistent Volume.

Mounting a persistent Volume claim as a Volume looks similar to the previous Volumes you encountered.

You can define the Volume in your Pods, and then you can mount it inside a container on a particular path:



pod.yaml

```
kind: Pod
apiVersion: v1
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
      - mountPath: "/var/www/html"
        name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: myclaim
```

When the Persistent Volume Claim is mounted in the in Pod, the volume is attached to the Node first and then made available to all containers.

Until the Persistent Volume Claim is used into a Pod, the disk is unmounted.

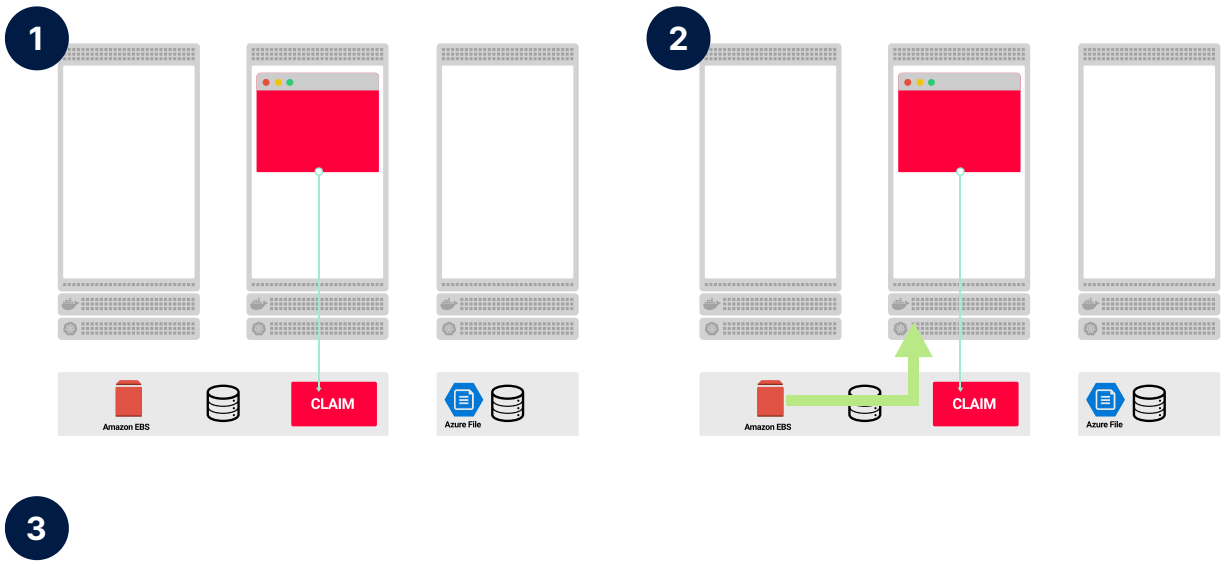


Fig. 1 You can use a Persistent Volume Claim as a Volume in your Pod.

Fig. 2 The Persistent Volume Claim is bound to a Persistent Volume. When the Pod requests the storage, the volume is mounted to the Node first.

Fig. 3 After the Volume is mounted into the Node, it can finally be mounted inside the Pod as well.

The Volume mapped by the Persistent Volume is only mounted to the Node when the Pod is created.

There's no need to mount the Volume if there's no Pod requesting it.

Having to map Persistent Volume to storage resources manually is a full-time job if you are working in a large team.

Kubernetes has a solution for this, and it's called dynamic provisioning of Volumes.

Dynamically provisioning of Persistent Volumes

The idea is smart: instead of having to create Persistent

Volume and Volumes manually, you could have a mechanism to create both every time a Persistent Volume Claim is created.

You could define a list of templates for Persistent Volumes:

- A "standard" storage option with 40GiB backed by Elastic Block Store on Amazon Web Services
- A "local" storage option with 5GiB backed by hostPath to use as a scratch disk
- A "shared" storage option with 100GiB backed by Ceph that can be read and written by multiple Nodes and Pods

When a Persistent Volume Claim is created with a choice from that list, instead of waiting for a Persistent Volume to be available Kubernetes uses the template to provision a Persistent Volume of that type waiving the burden of manually provisioning storage by yourself.

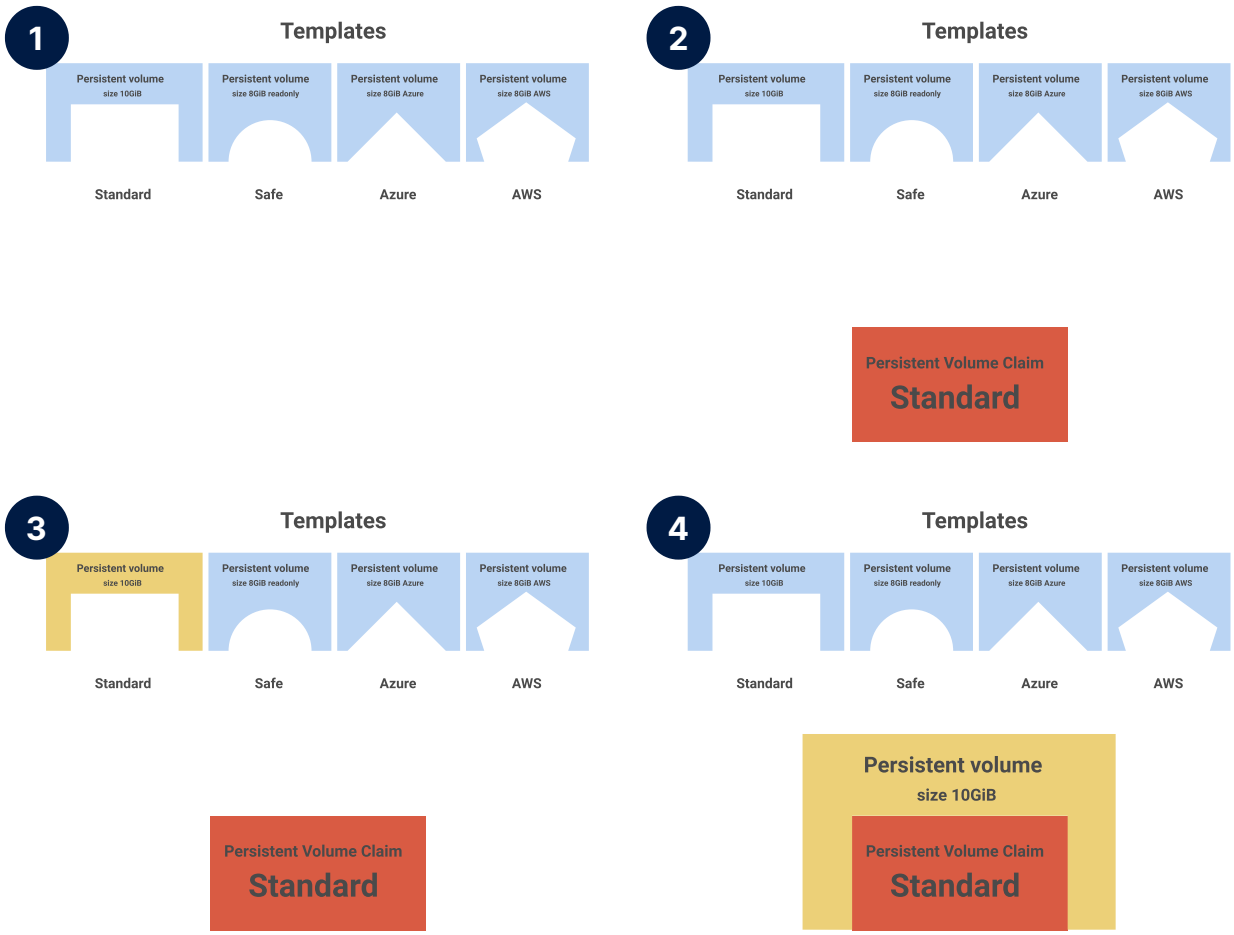


Fig. 1 You can define a set of templates for your Volumes and Persistent Volumes. Perhaps you have *fast* and expensive and *slow* and cheap storage. You could create a recipe for each of them and give them a name.

Fig. 2 A Persistent Volume Claim is a request for storage. When you make a request, you specify the template you wish to use.

Fig. 3 Kubernetes creates the Volume in your cloud provider and associated Persistent Volume as described in the template.

Fig. 4 The Persistent Volume Claim can finally bind to the Persistent Volume created.

Depending on where your Kubernetes cluster is hosted, you might have access to different StorageClasses.

If you use minikube, the default StorageClass is "standard" and uses hostPath.

You can provision a Persistent Volume Claim with a storageClass like this:

pvc.yaml

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
```

Using local volumes as an alternative for hostPath

When you learnt about hostPath, you discovered that if you want your Pod to reuse the same directory on the same Node, you might need to intervene and manually schedule the Pod.

You're not guaranteed that the Pod is rescheduled on the same node all the time.

But being able to use the filesystem on the node is convenient, particularly if you have an application that can replicate the data across Nodes.

If the app is intelligent enough to reconcile the differences with the other instances, it doesn't matter that the folder on one of the nodes is momentarily out of sync.

You still face the challenge of scheduling, though.

Even if your app can replicate and reconcile the data, you could have a Pod landing on a new node that perhaps wasn't mean for storing files.

It turns out that having applications with terabytes of data that have to host data locally and replicate across nodes is a popular request.

Famous examples include distributed data stores like Cassandra, or distributed file systems like Gluster or Ceph.

For that reason, you can use the local volume.

When you define a local volume as a Persistent Volume, you can select which node the Persistent Volume is attached to.

The scheduler will schedule Pods only to those nodes which can support the Persistent Volume Claim associated with

that Persistent Volume.

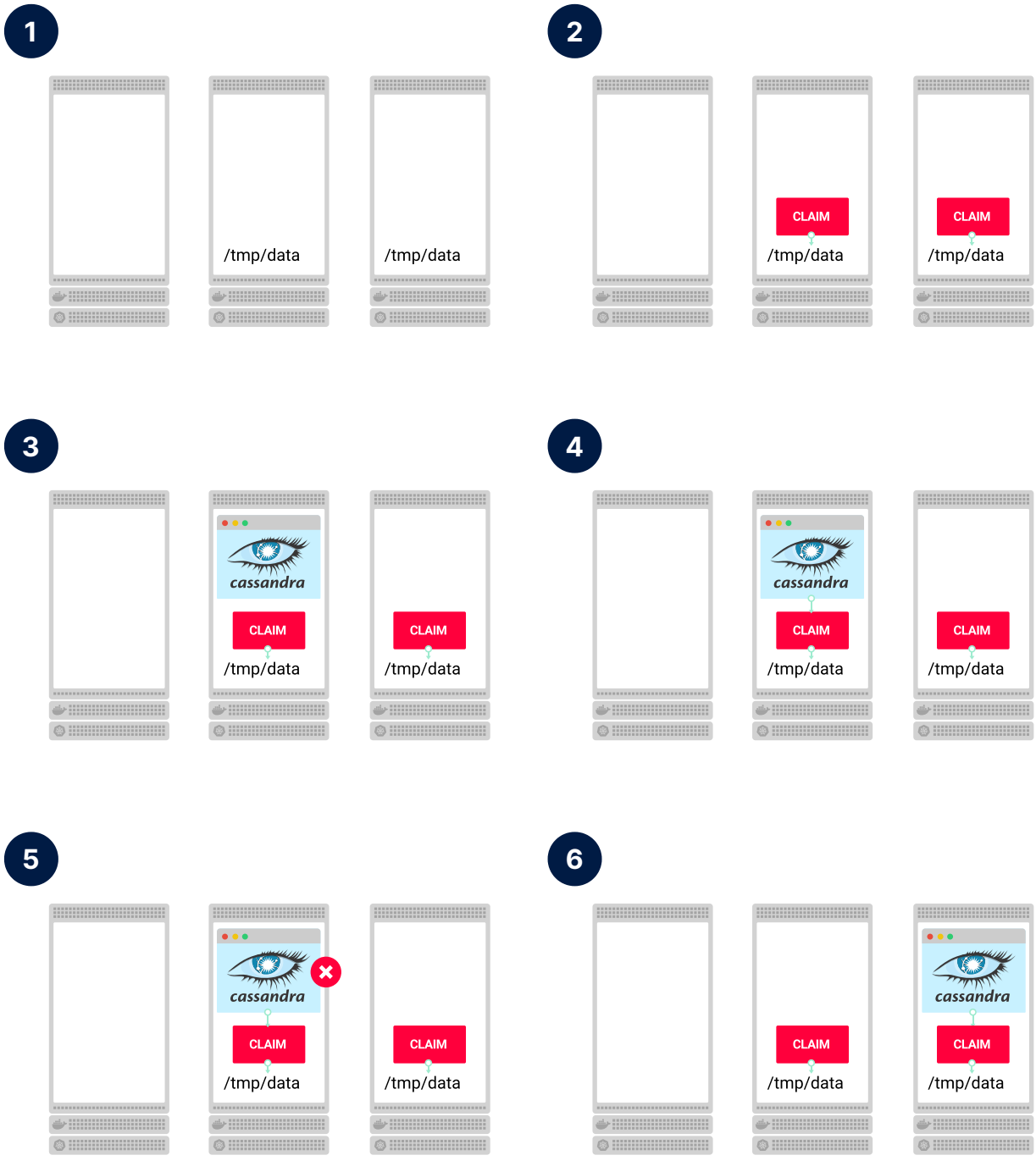


Fig. 1 You can define a Persistent volume to use a directory on the Node using local volumes.

Fig. 2 You can manually map Persitent Volume Claims to those Persitent Volumes — unfortunately dynamic provisioning isn't available here.

Fig. 3 You could deploy an application able to replicate data across Nodes such as Cassandra to use the local volume.

Fig. 4 You could deploy an application able to replicate data across Nodes such as Cassandra to use the local volume.

Fig. 5 If the Pod with Cassandra is deleted, Kubernetes reschedules it somewhere with the right Persistent Volume Claim.

Fig. 6 The Cassandra Pod will never land on the leftmost Node.

Persistent Volumes for local volumes have to be created in advanced, and the corresponding Persistent Volume Claim has to be linked manually to point to the Persistent Volume.

You can't use dynamic provisioning with StorageClass to automatically request local volumes.

Also, a local volume doesn't solve the challenge of losing Nodes and thus the data.

If a Node becomes unavailable, all the data stored in that node is lost.

Your application should handle that occurrence when it happens.

Mounting custom Volume drivers

All volumes you discovered so far are "in-tree" meaning they are built, linked, compiled, and shipped with the core Kubernetes binaries and extend the core Kubernetes API.

Adding a new volume requires creating a Pull Request against the official Kubernetes repository, propose the change, hope it is accepted and merged and then wait for the next release of Kubernetes.

If you want to create a new volume because you have an idea about mounting Hashicorp Vault secrets as files in Pods, it could take months if not years to have your plugin materialised in the next release of Kubernetes.

And that's if it's accepted into the project.

Having volume drivers hardcoded in the project doesn't scale.

Originally flexVolumes and more recently the CSI are two ways to create dynamic volume drivers that are not shipped by the Kubernetes project.

You're probably familiar with the term CSI.

When you learnt about the `kubelet`, you discovered that it executes three critical tasks when a Pod is created on a Node.

The `kubelet`:

- Creates the Pod by delegating the task to the container runtime — most of the times Docker
- Delegates attaching the container to the network to the container network interface
- Attaches any volumes to the Pod. In the case of the Flexvolume and CSI, it delegates that task too

The CSI is meant to replace FlexVolumes as a more abstract and comprehensive solution. But there're a few favourite volumes plugins built on top of FlexVolumes.

The Azure/Kubernetes-KeyVault-Flexvolume project from Azure is one of such projects. Once installed in your cluster, you can mount secrets from Azure keyvault as files in your Pods.

It's not hard to imagine that one day all vendor-specific plugins could be extracted from the Kubernetes codebase and released as standalone drivers.

Designing a clustered PostgreSQL

Now that you are familiar with Volumes, Persistent Volumes, Persistent Volume Claims, Deployments and Services, let's see how you can leverage those to create a distributed and scalable PostgreSQL cluster.

Instead of starting from designing the cluster in Kubernetes, you should start from the basics.

In a clustered PostgreSQL you have a primary node and few secondaries, and ideally, you have an odd number of instances.

The secondary instances have to boot after the primary; otherwise, they can't replicate.

All of the three instances have to be backed by persistent storage.

You can't restart one replica and having it losing the entire data.

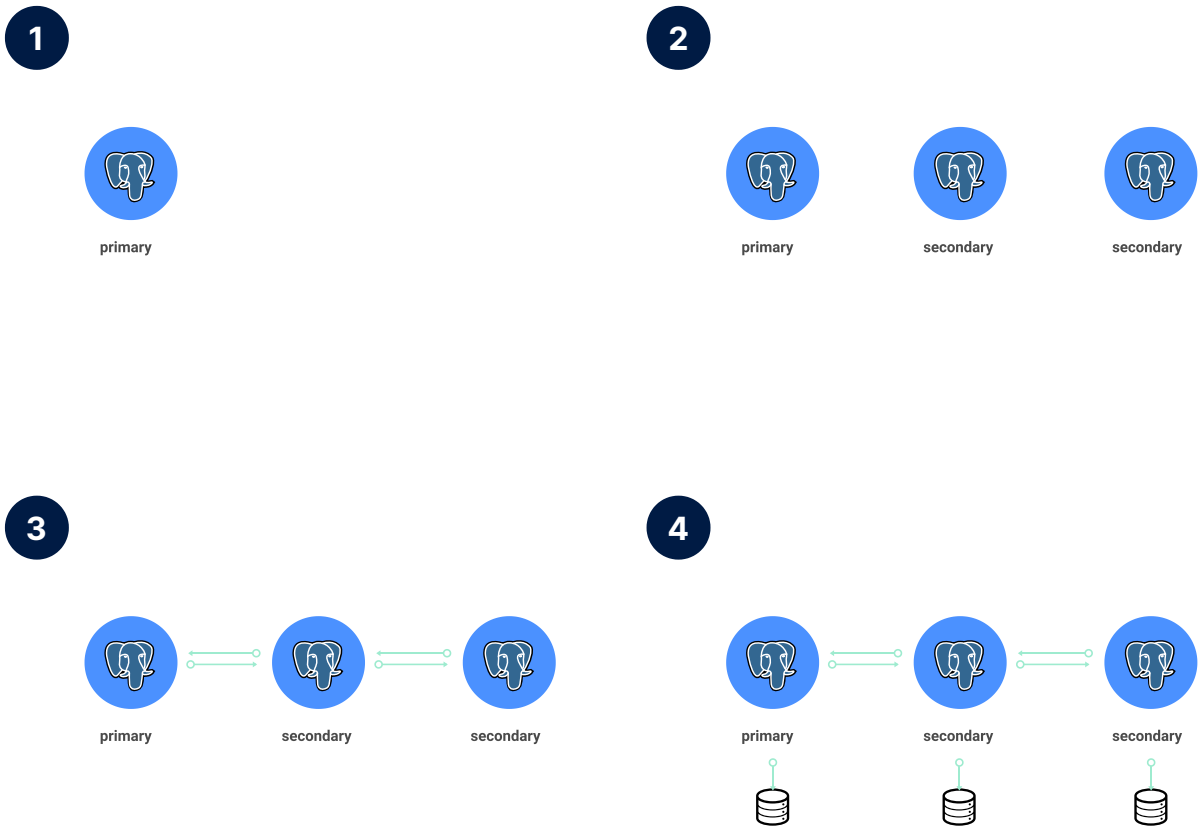


Fig. 1 In a clustered PostgreSQL you should have a primary instance.

Fig. 2 Once the primary is booted, you can start more secondary instances and replicate the data.

Fig. 3 Once the primary is booted, you can start more secondary instances and replicate the data.

Fig. 4 You should persist the data and have a storage mechanism.

Now that you have a plan let's see what Kubernetes has to offer.

Each instance of the database could be wrapped into a Pod.

Since there are three Pods, this suggests that you could create a Deployment with three replicas.

Pods have to talk to each other to form a cluster.

So perhaps you could create a Service that is pointing to the same label as the Deployment.

Finally, you need to provision for Persistent Volumes, and Persistent Volume Claims to make sure that the data is stored safely on disk.

You could use the local volume for that.

The final bill of material is the following:

- A Deployment with three replicas for the three instances: 1 primary, 2 secondary instances of PostgreSQL
- A Service to connect the Pods
- 3 Persistent volumes with a local Volume and 3 Persistent Volume Claims to store the data

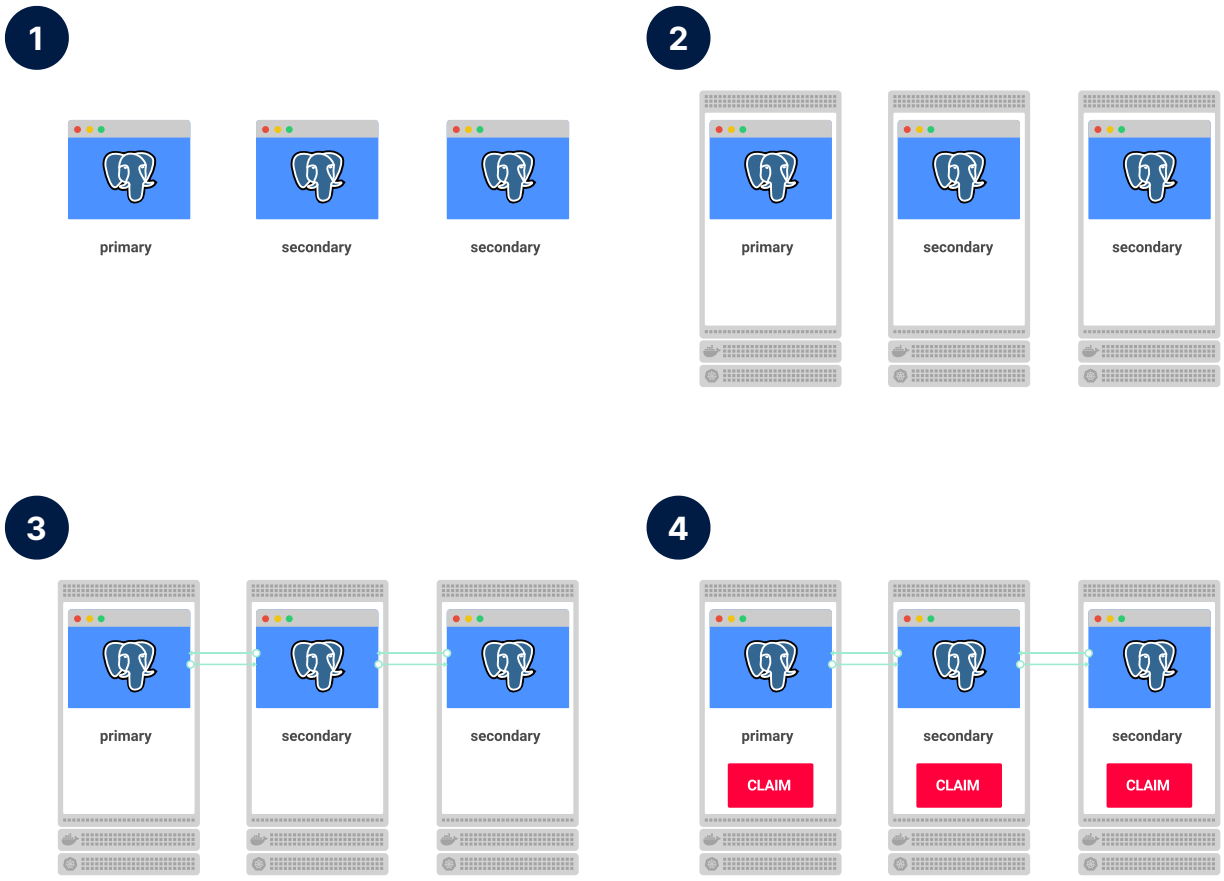


Fig. 1 In Kubernetes, you could have your primary and secondaries as Pods.

Fig. 2 You could use a Deployment to manage the Pods and make sure they respawn when they're deleted.

Fig. 3 You should create a Service so that you can route traffic between Pods and trigger the replication.

Fig. 4 You should use Persistent Volumes and Persistent Volume Claims to persist the data.

It's time to test the setup.

So you create the resources and observe what happens.

You should immediately notice something annoying.

The Deployment isn't playing nicely and creates the Pods all at the same time.

And since it's a race, sometimes the first node is launched first, sometimes is the third.

It's not predictable.

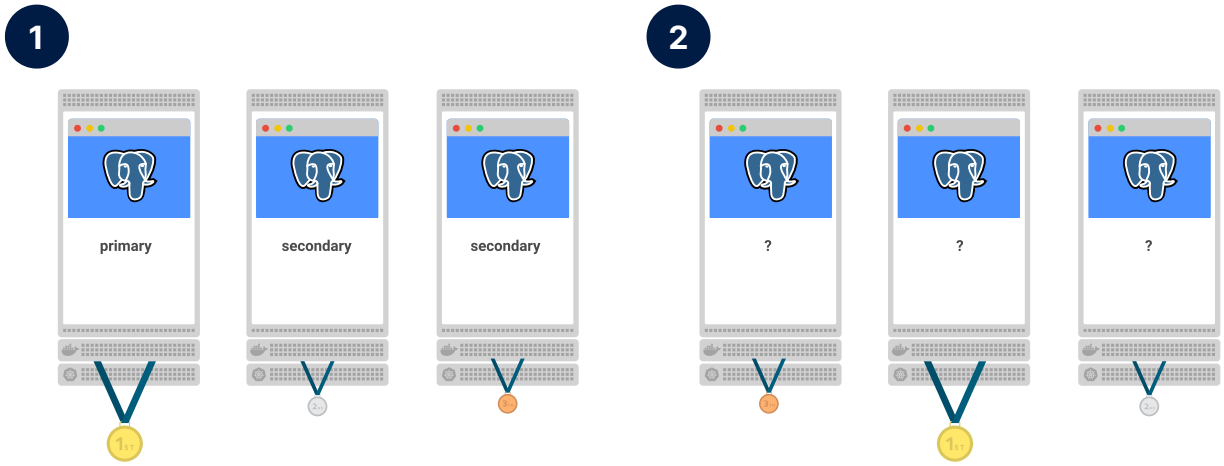


Fig. 1 The order in which the Pods are launched is not predictable when you use a Deployment.

Fig. 2 The order in which the Pods are launched is not predictable when you use a Deployment.

That doesn't play well with the database because you don't know who ends up being the primary.

The second annoyance comes from the secondary instances.

When they connect to the Service to form a cluster, the Service directs the traffic to one of the three instances.

It doesn't route the traffic just to the primary.

After all the Service is a load balancer.

So the cluster becomes such only when you're lucky, and both secondaries are routed to the primary node (side note, if you manage to do that, you should buy a lottery ticket).

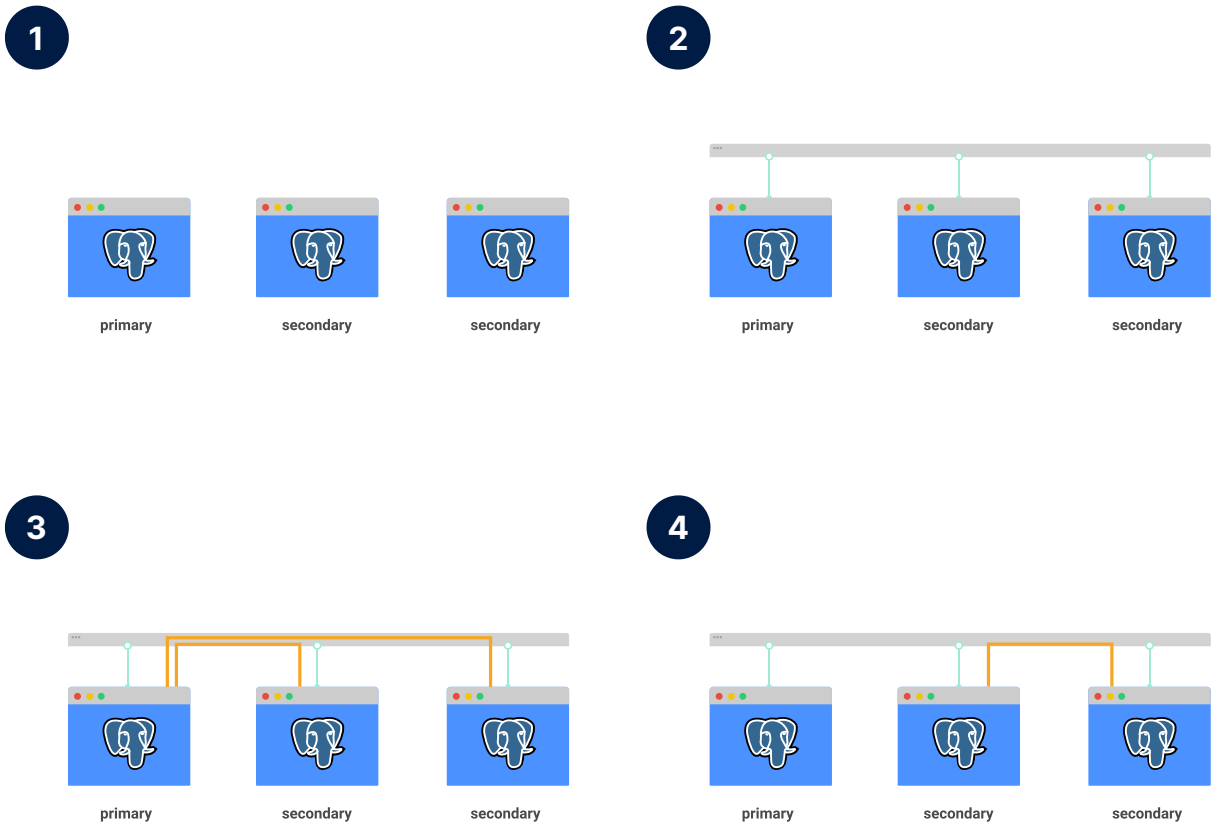


Fig. 1 You have a Deployment with three replicas.

Fig. 2 You also provisioned a Service to route the traffic to three instances.

Fig. 3 Ideally, when the secondary request to talk to the primary, they go through the Service and finally reach the primary database.

Fig. 4 However, the Service doesn't know who is the primary and routes the traffic to any Pod that can find.

Deployment and Service are letting you down this time.
What you should have instead is:

- Ordered bootstrapping. Only create the secondary after the primary
- Both secondaries should be able to connect to the primary
- Persistence was great, and you should keep it that way

Kubernetes has a specific object to deal with the above requirements: the StatefulSet.

Creating a clustered database using StatefulSets

A StatefulSet is akin to a Deployment; both can define a set of Pods. The StatefulSet though has some characteristics that are unique to it:

- You can decide the order in which the Pods are created
- The naming convention for the Pods doesn't contain gibberish such as `postgresql-86c58d9df4-c4r5g`. Each Pod is numbered incrementally from zero. In this example, you can expect to have `postgresql-0`, `postgresql-1`, and `postgresql-2` as Pod names.
- With an extra headless Service, you can configure to have the name of the Pod addressable as a DNS entry. In other words, you can connect to the first pod with `postgresql-0`.
- Each Pod has its Persistent Volume Claim and stable, persistent storage
- When a Pod is deleted, the StatefulSet controller recreates the Pods and attaches it to the previous Persistent Volume Claim

That sounds exactly what you need to create your clustered PostgreSQL.

You could replace the Deployment with a StatefulSet and the Service for a Headless Service.

The new setup works differently now.



Fig. 1 Consider the following three Nodes cluster. When you use a StatefulSet, Kubernetes create the primary database first and waits for it to be *Running*.

Fig. 2 Once the primary is *Running*, it creates another Pod. As part of the boostrapping that Pod can talk to the primary using its DNS entry `pod-1.postgresql.default.svc.cluster.local` and start replicating.

Fig. 3 Once the second Pod is *Running*, Kubernetes launches the third and waits for it to be available.

Fig. 4 All the Pods are backed by Persistent Volume Claims and Persistent Volumes.

Fig. 5 All the Pods have a DNS entry with their name.

What happens when a Pod is deleted or removed?

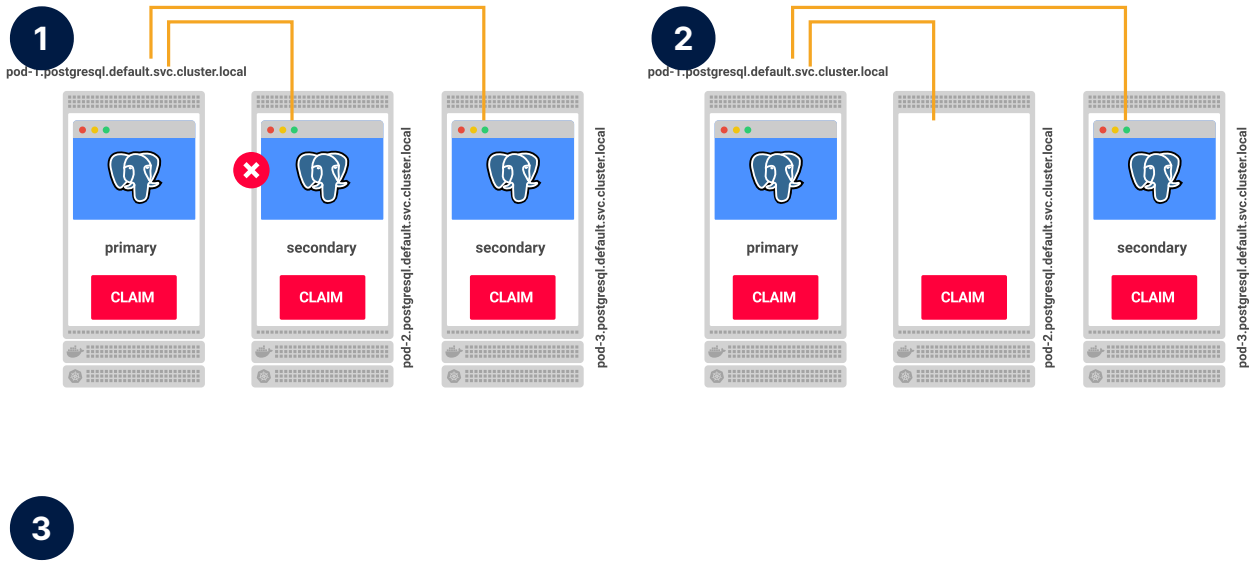


Fig. 1 Let's delete the second Pod.

Fig. 2 *What does Kubernetes do?*

Fig. 3 When a Pod is deleted, Kubernetes recreates the Pod and attaches it to the same Persistent Volume and Persistent Volume Claim.

When the Pods complete the replication, the cluster is entirely bootstrapped and ready to receive traffic.

You have fully distributed, clustered PostgreSQL setup that is resilient to failures.

But is it resilient to failures?

Consider what happens when the primary Pod goes down.

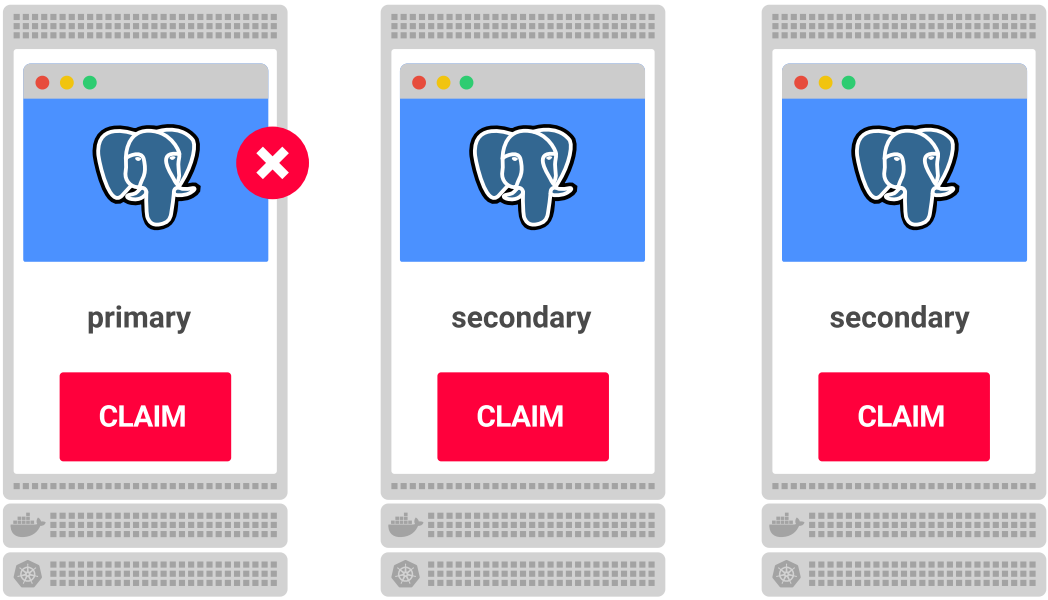


Fig. A Pod is deleted from a StatefulSet

Ideally:

- one of the two secondary becomes the primary
- the StatefulSet recreates the Pod
- The Pods joins as a secondary and starts replicating with the primary

If the order of actions above is respected, the database can

continue receiving read and write requests even if one of the instances of the cluster is lost.

The assumption is that the StatefulSet is smart enough to wait for the leader election to occur before creating the new Pod.

Unfortunately, that doesn't happen.

The StatefulSet isn't aware that you decided to run a clustered PostgreSQL.

So it creates the Pod as soon as possible.

If it brings the Pod back with a slight delay, it could be that one of the secondaries was already elected to be the leader.

And you could end up with two instances believing to be the primary.

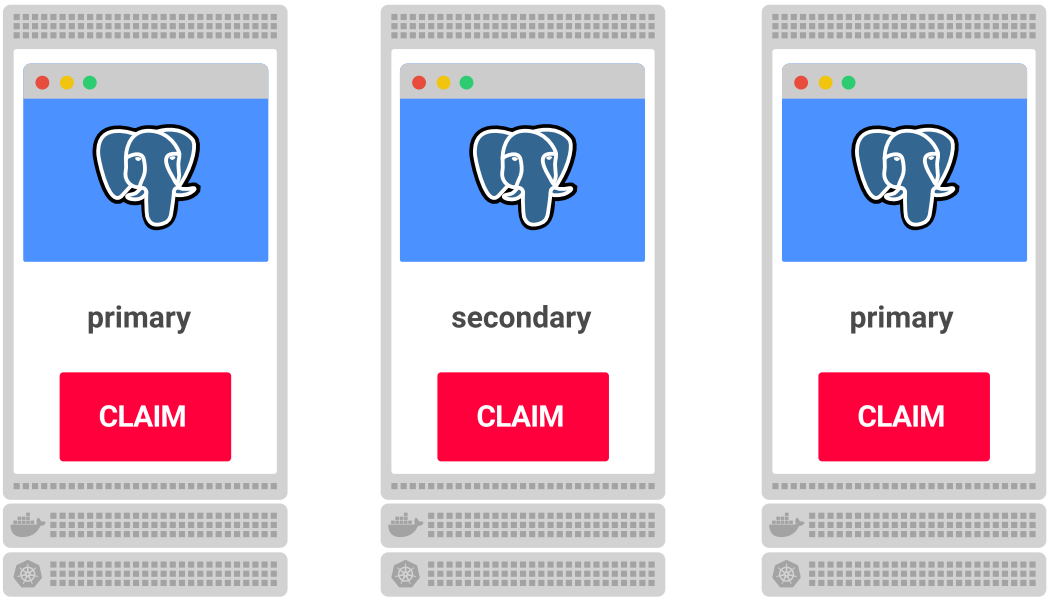


Fig. Two instances as primary

That's a nightmare scenario for a clustered database.

However, that is the default behaviour of the and StatefulSet and StatefulSet controller.

But what if you could write your mechanism to control and managed Pods?

How the StatefulSet controller works

The controller manager watches the API for StatefulSet objects, creates the Pods and Persistent Volume Claims and keeps monitoring them.

When a Pod is lost, the StatefulSet controller inside the controller manager recreates it.

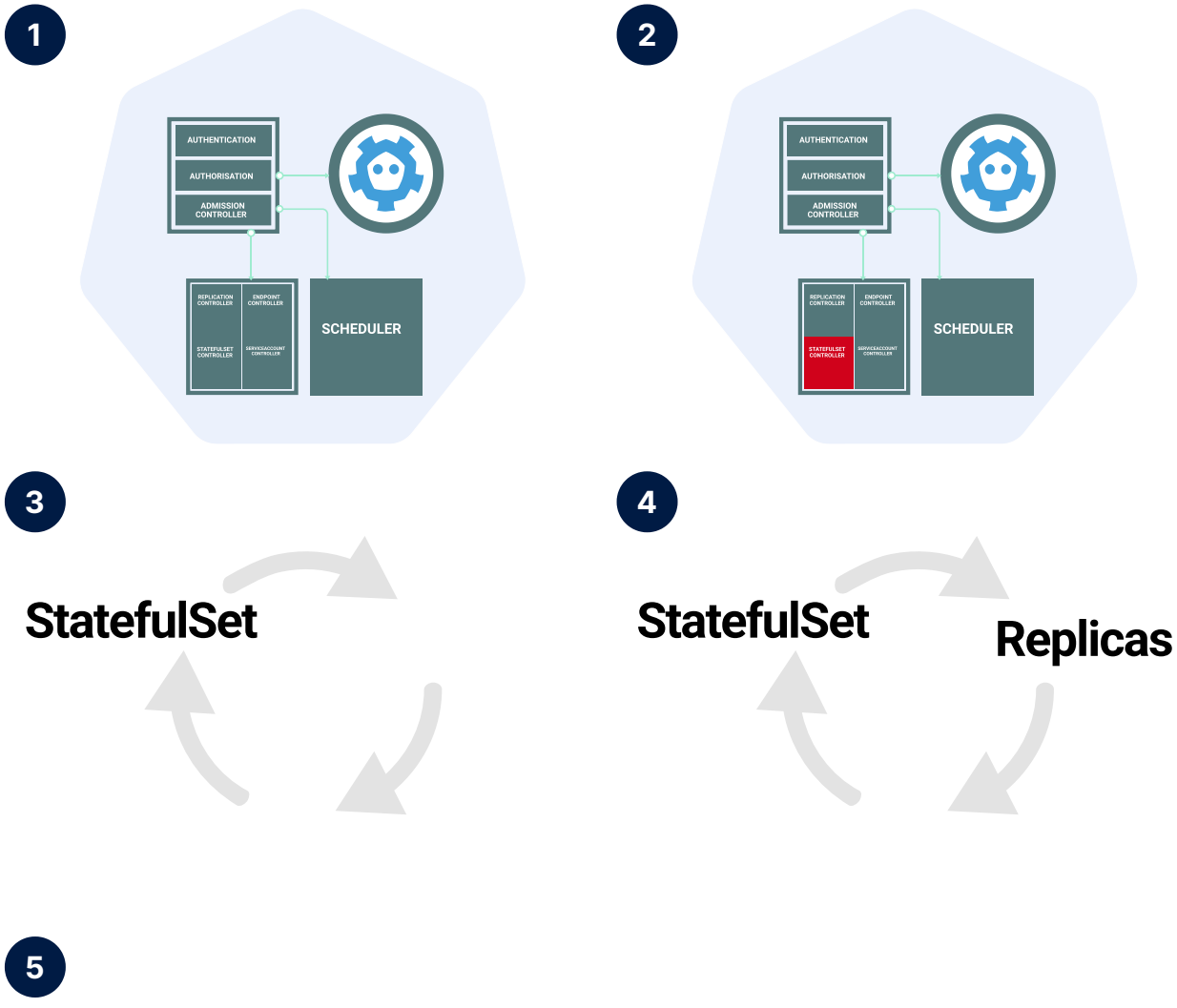


Fig. 1 Inside the control plane, the controller manager is in charge of watching APIs and creating objects.

Fig. 2 The StatefulSet controller is part of the controller manager and is in charge of monitoring StatefulSet objects.

Fig. 3 When you create a StatefulSet manifest with `kubectl apply -f statefulset.yaml` the YAML is sent to the API and stored in etcd.

Fig. 4 The StatefulSet controller inside the controller manager detects the new resource and creates the Pods, Persistent Volume and Persistent Volume Claims.

Fig. 5 The StatefulSet controller inside the controller manager detects the new resource and creates the Pods, Persistent Volume and Persistent Volume Claims.

That's very similar to another controller you're familiar with: the Ingress controller.

Similarly, the Ingress controller watches for Ingress manifests, writes the virtual host rules on a file and hot-reloads a web server.

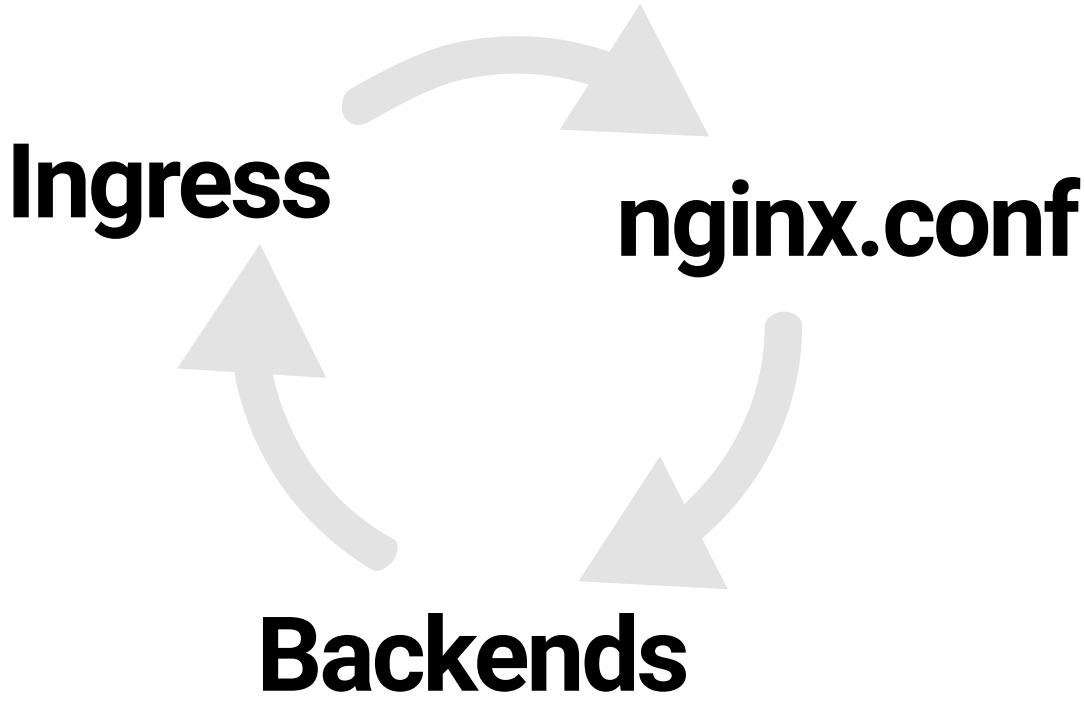


Fig. The Ingress controller

The controller subscribes for updates to manifests and endpoints — if one of those changes, it repeats the process and reconfigures the web server.

Those controllers could be part of the Controller Manager such as Endpoint Controller and StatefulSet controller, or they could be independent Deployment such as the Ingress controller.

What they have in common is the ability to talk to the Kubernetes API and

- Watch for *any* kind of object
- Create and destroy resources

Nothing stops you from doing the same.

Creating a controller for a clustered database

Imagine you could write some software that interacts with the Kubernetes API to create a Pod.

That's a POST request to the Rest API to the Pod Endpoint `/api/v1/namespaces/default/pods`.

You could create an instance of a PostgreSQL database.

Since you're in charge now, you could monitor the API and wait until the Pod is ready.

As soon that's done, you could create a second Pod.

And you could have that Pod replicating to the primary instance of the database.

And nothing is stopping you from doing the same with a third Pod.



Fig. 1 A controller is an application that can talk to the Kubernetes API. You could write your own and deploy it as a Deployment in your cluster.

Fig. 2 You could have your operator submitting a resource definition for a Pod such as a PostgreSQL Pod.

Fig. 3 You could monitor for the Pod to be *Running* before creating another Pod.

Fig. 4 And you could create another Pod.

Fig. 5 You could also create a Service using the same operator to route traffic to one of the instances.

In other words, you just recreated the StatefulSet controller as a separate component.

But why stop there?

You could have your controller monitoring the Pods.

If one goes down, you could take note of that and wait for a leader to be elected.

When there's a leader, you could request the API to create a Pod and have it replicated to the new primary.

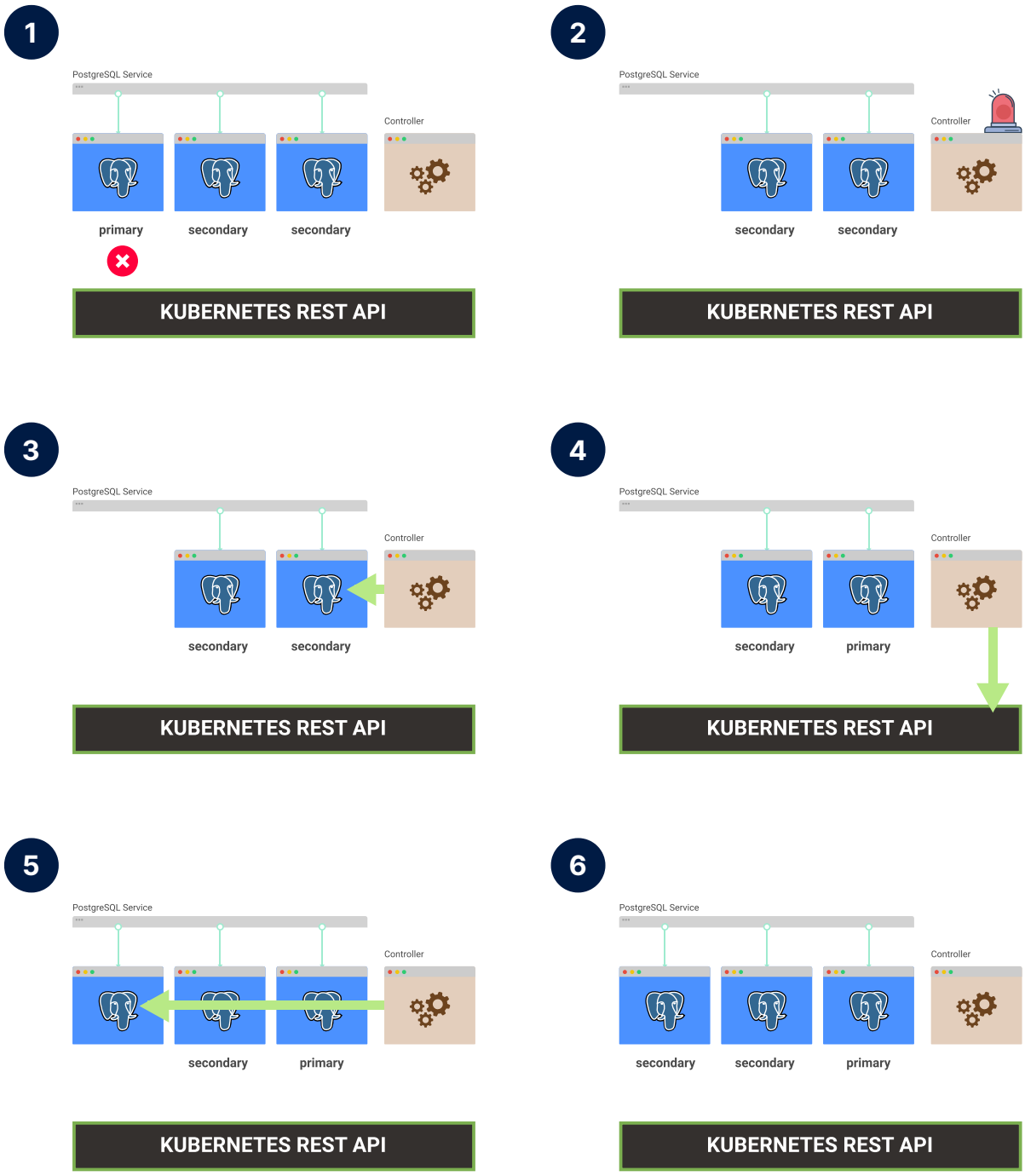


Fig. 1 Your operator could subscribe to changes to the Pods that were created.

Fig. 2 If one of the Pods is deleted, the controller receives a notification.

Fig. 3 Since you developed your controller to execute your logic, you could have the operator to trigger a leader election between the existing PostgreSQL instances.

Fig. 4 When the leader is elected, the operator could create the missing Pod.

Fig. 5 Finally, when the Pod is *Running*, you could have the operator to trigger the replication.

Fig. 6 The cluster is back to normal.

This is much better than a StatefulSet.

Your software can correctly handle failures and recover autonomously.

The logic is specific to this database and probably can't be used with another database such as Redis.

So perhaps there're good uses case for both: your software and a StatefulSet.

The piece of software you just wrote is often referred to as a controller or an operator.

There're a lot of operators in the wild, and many more will come during the years.

Some controllers are designed to interact with the Kubernetes API solely.

Those are usually referred to as controllers.

Others are most sophisticated and not only they can interact with the API but can register new kind of objects.

Those are usually referred to as Operators.

Most of the time you will find people using the terms operators and controllers interchangeably.

Database operators

Speaking of operators and databases, have you ever seen a Postgres kind?

postgres.yaml

```yaml
apiVersion: kubedb.com/v1alpha1
kind: Postgres
metadata:
  name: p1
  namespace: demo
spec:
  version: 9.6.5
  replicas: 1
  doNotPause: true
```

This is not a native Kubernetes object.

The Postgres kind is a Kubernetes extension created by the kubedb/operator project and is designed to encapsulate the details of running a clustered PostgreSQL database.

Like the operator described earlier.

The KubeDB operator is in charge of reading the specification and creating the Pods and Services necessary to set up the cluster.

Another popular operator is the awslabs/aws-service-operator operator.

Have you ever seen an Amazon Relational Database Service (RDS) kind?

What about an Amazon DynamoDB?

dynamodb.yaml

```yaml
apiVersion: service-operator.aws/v1alpha1
kind: DynamoDB
metadata:
  name: dynamo-table
spec:
  hashAttribute:
    name: name
    type: S
  rangeAttribute:
    name: created_at
    type: S
  readCapacityUnits: 5
  writeCapacityUnits: 5
```

Amazon Web Services created an operator that acts as a bridge between Kubernetes and their services.

When you create a DynamoDB kind, the AWS operator detects that a resource was created in etcd and provisions a real DynamoDB database.

When the table is ready, the operator calls the Kubernetes

API and creates a Secret with the credentials to connect to the database.

You can use the Secret to inject environment variables in your Pods (or mount it as a Volume).

Distributed storage with operators

Another exciting application of controllers and operators is distributed storage.

When it comes to storage, the dream is to have storage as a unified layer that could span several Nodes.

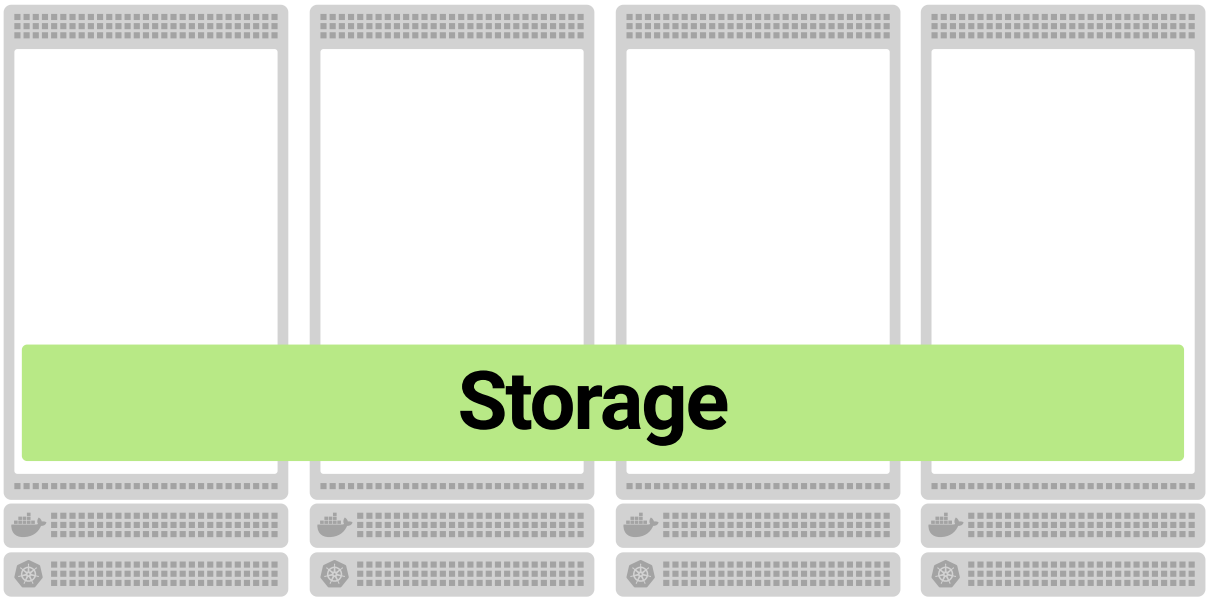


Fig. Storage that spans multiple Nodes

In reality that doesn't exist and you often find yourself patching several disks into a single layer via software.

OpenEBS is an excellent example of that.

In OpenEBS, Pods are in charge of mounting the local filesystem using hostPath.

Since hostPath isn't a reliable way to store data, OpenEBS has a mechanism to replicate the data across other Pods.

If one Pod had to die, the others have enough information to reconstruct the data.

The replication between Pods is managed and controlled by an operator.

The operator is also in charge of launching and monitoring the Pods as well as registering a StorageClass in Kubernetes.

By doing so, OpenEBS is creating a unified volume that spans multiple Nodes by leveraging storage on the local disk mounted as containers.



Fig. 1 In OpenEBS you have Pods that mount the local filesystem on the Node as a hostPath Volume.

Fig. 2 hostPath has a number of drawbacks. One of the being unable to replicate the data across Nodes.

Fig. 3 OpenEBS has developed an operator that creates the Pods, monitors them and triggers the replication. Even if a Node is lost or added, the Pods can still continue working.

Since the Pods are used only to mount the filesystem and replication, they are referred to as storage replicas.

Those Pods are then exposed to Kubernetes as Persistent Volumes through a StorageClass.

Using OpenEBS to launch a clustered database

At this point, nothing is stopping you from launching a PostgreSQL cluster with three replicas that are backed by Persistent Volume Claims using the OpenEBS volumes.

Please notice how everything in the cluster is running in

Pods: from the storage replicas to the database instances.

So what if one of the database Pod where to die?

The clustered database operator will take care of triggering replication and leader election.

What if one of the storage replicas goes down?

The database Pod will be broken as well.

Thankfully to OpenEBS, the data isn't stored in one place.

The OpenEBS operator creates a new Pod and triggers replication of the data.

As soon as the Pod is ready, the database Pod can use the Persistent Volume claim to use the Persistent volume and come back up.

You can imagine that a similar scenario applies if the Node is lost.

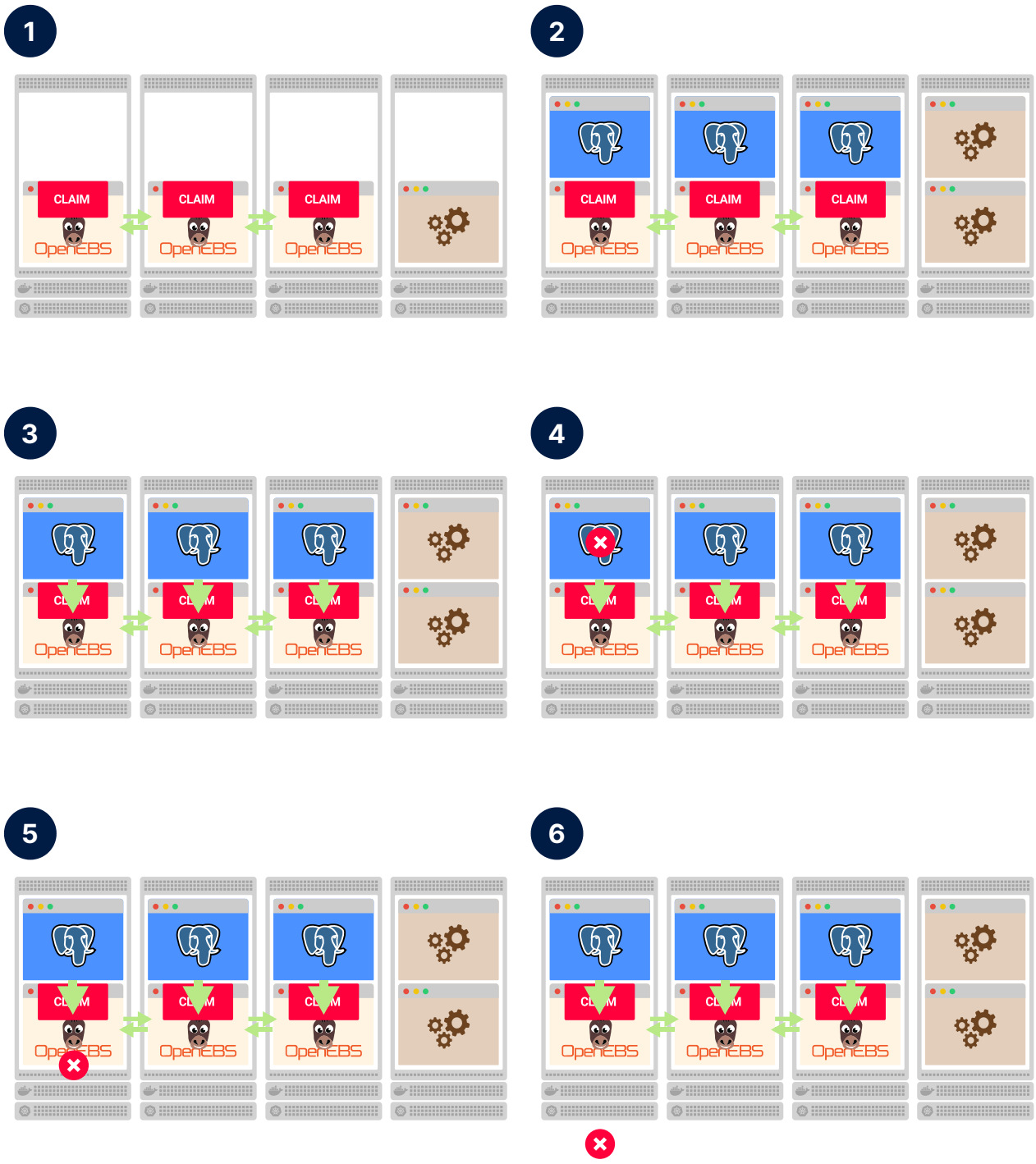


Fig. 1 OpenEBS exposes the Pods as a StorageClass. You could create a Persistent Volume Claim and request for a dynamic Volume backed by OpenEBS.

Fig. 2 You could create your own PostgreSQL cluster with operator and use the Volumes provided by OpenEBS.

Fig. 3 You could create your own PostgreSQL cluster with operator and use the Volumes provided by OpenEBS.

Fig. 4 When a Pod is deleted, the PostgreSQL operator triggers the replication as discussed earlier.

Fig. 5 When a storage replica is lost, the database Pod is lost too. The OpenEBS and PostgreSQL operators trigger their procedure for recovering from a lost Pod.

Fig. 6 Similarly, if you lose a Node, the operators trigger the chain of events necessary to stabilise the deployment.

The data is still replicated twice: in OpenEBS and PostgreSQL.

Which also raised a question: is replicating the information twice necessary and efficient?

Depending on what you do, you might be able to use hostPath and get away with replication on PostgreSQL.

You should also notice that there're several components involved in this diagram:

- Replication of the filesystem inside containers
- Database inside containers reading a filesystem from a

- container
- Two controllers managing replicas

Running stateful workloads in Kubernetes isn't easy.

You should pay attention to how many components are involved and what's the risk of one of them going down.

If you are risk-averse and wish to leverage proven solutions, using operators such as the AWS operator or the Service catalogue might be a better solution than rolling out your clustered database or unified storage layer.

Chapter 2

Deploying a stateless MySQL

Storage is a critical part of running stateful containers, and Kubernetes offers powerful primitives for managing it.

Dynamic volume provisioning, a feature unique to Kubernetes, allows storage volumes to be created on-demand.

The challenge with containers is that files written within the container are ephemeral and lost when the container stops.

In this exercise, you will use persistent volumes to add non-ephemeral storage in Minikube to store the MySQL database.

Prerequisites

You should have installed:

- minikube
- kubectl and
- MySQL Workbench

Creating the Deployment

You should use the MySQL container from the public Docker Hub registry to create your Deployment.

The official documentation suggests that you can customise the credentials for the MySQL with few environment variables.

You can use

- `MYSQL_ROOT_PASSWORD` to customise the root password
- `MYSQL_DATABASE` to pick the name of the database
- `MYSQL_USER` and `MYSQL_PASSWORD` to customise the username and password used to connect to the database

With those parameters, you can create a `deployment-mysql.yaml` with the following content:

deployment-mysql.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
  labels:
    name: mysql-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      name: mysql
  template:
    metadata:
      labels:
        name: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0.2
```

```
          ports:
            - containerPort: 3306
              protocol: TCP
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: password
            - name: MYSQL_DATABASE
              value: sample
            - name: MYSQL_USER
              value: mysql
            - name: MYSQL_PASSWORD
              value: mysql
```

You can create the resource with:

bash

```
$ kubectl apply -f deployment-mysql.yaml
```

You can expose the deployment with a Service.
Create a `service-mysql.yaml` with the following content:

service-mysql.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  ports:
  - port: 3306
    name: mysql
    targetPort: 3306
    nodePort: 31000
  selector:
    name: mysql
  type: NodePort
```

You can create the resource with:

bash

```
$ kubectl apply -f service-mysql.yaml
```

Storing credentials in Secrets

It's good practice to not store credentials such as usernames and passwords in version control.

You should refactor the Deployment and extract the credentials into a Kubernetes Secret.

A Secret is a resource that holds key-value pairs.

Create a `secret.yaml` with the following content:

secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  username: bXlzcWw=
  password: bXlzcWw=
  root: cGFzc3dvcmQ=
```

> Please note how the values are base64 encoded. Encoding the values as base64 means, you can include binaries such as certificates.

You can create a Secret in Kubernetes with:

bash

```
$ kubectl apply -f secret.yaml
```

You should update your Deployment to reference the credentials in the Secret.

deployment-mysql.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
  labels:
    name: mysql-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      name: mysql
  template:
    metadata:
      labels:
        name: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0.2
          ports:
            - containerPort: 3306
```

```
            protocol: TCP
        env:
          - name: "MYSQL_ROOT_PASSWORD"
            valueFrom:
              secretKeyRef:
                name: mysecret
                key: root
          - name: "MYSQL_DATABASE"
            value: "sample"
          - name: "MYSQL_USER"
            valueFrom:
              secretKeyRef:
                name: mysecret
                key: username
          - name: "MYSQL_PASSWORD"
            valueFrom:
              secretKeyRef:
                name: mysecret
                key: password
```

You can update the Deployment with:

bash

```
$ kubectl apply -f deployment-mysql.yaml
```

Verifying that you can connect to MySQL

Download MySQL Workbench, or your favourite MySQL GUI tool.

You can retrieve the IP address of the cluster with:

bash

```
$ minikube ip
```

You should be able to connect to MySQL with your favourite admin interface and the following credentials:

Credentials

```
Host: "replace with minikube ip"
Username: root
Password: password
Database: sample
Port: 31000
```

Go ahead an create a table.

SQL

```
CREATE TABLE pet (name VARCHAR(20));
```

Congratulations, you deployed a MySQL database and created a database with a schema.

Chapter 3

Persisting changes

What happens when you delete the MySQL Pod?
Is the table you created still available?

You should explore what happens when you delete the MySQL Pod.

You can list all running Pods with:

```bash
$ kubectl get pods
```

You can delete the MySQL Pod with:

```bash
$ kubectl delete pod <replace with Pod name>
```

The Kubernetes scheduler creates a new Pod:

```bash
$ kubectl get pods
```

You should connect to the newly created pod with Sequel Pro and look for the `pet` table.

The table doesn't exist anymore.

Any time you restart (or the Pod crashes) the state is lost.

You can mount a volume to persist the changes to disk.

Volumes in Kubernetes have a different lifecycle from Pods.

Volumes outlive any container running in a Pod, and the data is preserved across container restarts.

At its core, a volume is just a directory with some data in it.

You can create a persistent volume (often abbreviated PV) that represent an abstraction over the kind of storage you want.

There're types of Persistent Volumes such as Amazon EBS, GCEPersistentDisk, Glusterfs, etc.

For simplicity, you're going to use the local filesystem.

Creating a Persistent Volume

Create a `pv.yaml` file with the following content:

pv.yaml

```yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: pv0001
  labels:
    type: local
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/somepath/data01"
```

And create the resource with:

```bash
$ kubectl apply -f pv.yaml
```

You can verify that the Persistent Volume was created correctly with:

```bash
$ kubectl get pv
```

Claiming a Persistent Volume

To use a Persistent Volume you have to claim it.
You should use a Persistent Volume Claim to declare that you wish to use a particular Persistent Volume.
Create a `pvc.yaml` with the following content:

```yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc
spec:
  storageClassName: ""  # Static provisioning
```

```
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

Let's create persistent volume claim using the above template.

bash

```
$ kubectl apply -f pvc.yaml
```

After creating the Persistent Volume Claim, your persistent volume will change the state from *Available* to *Bound*.
You can verify that with:

bash

```
$ kubectl get pv,pvc
```

Let's use the Persistent Volume Claim to mount a volume in your MySQL Deployment.

Using volumes in Deployments

Edit your `deployment-mysql.yaml` file to include the `volumes` section:

deployment-mysql.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
  labels:
    name: mysql-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      name: mysql
  template:
    metadata:
      labels:
        name: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0.2
          volumeMounts:
          - mountPath: /var/lib/mysql
            name: mysqlvolume
          ports:
            - containerPort: 3306
              protocol: TCP
          env:
            - name: "MYSQL_ROOT_PASSWORD"
              valueFrom:
                secretKeyRef:
                  name: mysecret
                  key: root
            - name: "MYSQL_DATABASE"
              value: "sample"
            - name: "MYSQL_USER"
              valueFrom:
                secretKeyRef:
                  name: mysecret
                  key: username
            - name: "MYSQL_PASSWORD"
```

```
              valueFrom:
                secretKeyRef:
                  name: mysecret
                  key: password
      volumes:
        - name: mysqlvolume
          persistentVolumeClaim:
            claimName: test-pvc
```

You can update the deployment with:

bash

```
$ kubectl apply -f deployment-mysql.yaml
```

Testing the persistence

It's time to repeat the test you did earlier.

Connect to MySQL with your favourite admin interface and create a new table with:

SQL

```
CREATE TABLE pet (name VARCHAR(20));
```

Close the connection and list all the Pods with:

bash

```
$ kubectl get pods
```

You can delete the MySQL Pod with:



```bash
$ kubectl delete pod <replace with Pod name>
```

A new Pod is rescheduled.

If you connect to it again, you should be able to see the `pet` table.

Hurrah, persistence!

Chapter 4

Dynamic provisioning

Creating Persistent Volumes and Persistent Volume Claims is a lot of repetitive work.

You can leverage dynamic volume provisioning, a feature unique to Kubernetes, to provision on-demand persistent volume and storage.

Persistent Volume Claims can have a particular annotation where you can define a Storage Class.

The StorageClass is a representation of a specific type of storage that exists in the specified cloud provider.

It creates a level of abstraction from the cloud provider.

The StorageClass is very similar to the Adapter Pattern.

There're several implementations of the Storage Class such as Amazon EBS, GCEDisk, etc., and they all offer a compatible interface with StorageClass so that you don't have to worry about how the underlying resource is provisioned.

You can list the storage classes available on your cluster with:

bash

```
$ kubectl get storageclass
```

`minikube` has only one Storage Class that leverages the filesystem.

Creating volumes with a Storage Class

You can update your Persistent Volume Claim to use *standard* storage class.

Create a `pvc-standard.yaml` file with the following content:

pvc-standard.yaml

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-2
spec:
  storageClassName: "standard"
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

You can submit the definition with:

bash

```
$ kubectl apply -f pvc-standard.yaml
```

Notice how a Persistent Volume was created and attached to the Persistent Volume Claim:

bash

```
$ kubectl get pv,pvc
```

Let's create a second Deployment to test the Storage Class. Create a `deployment-mysql2.yaml` file with the following content:

deployment-mysql2.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment2
  labels:
    name: mysql-deployment2
spec:
  replicas: 1
  selector:
    matchLabels:
      name: mysql2
  template:
    metadata:
      labels:
        name: mysql2
    spec:
      containers:
        - name: mysql
          image: mysql:8.0.2
          volumeMounts:
          - mountPath: /var/lib/mysql
            name: mysqlvolume
          ports:
            - containerPort: 3306
              protocol: TCP
          env:
            - name: "MYSQL_ROOT_PASSWORD"
              valueFrom:
                secretKeyRef:
                  name: mysecret
                  key: root
```

```
            - name: "MYSQL_DATABASE"
              value: "sample"
            - name: "MYSQL_USER"
              valueFrom:
                secretKeyRef:
                  name: mysecret
                  key: username
            - name: "MYSQL_PASSWORD"
              valueFrom:
                secretKeyRef:
                  name: mysecret
                  key: password
      volumes:
        - name: mysqlvolume
          persistentVolumeClaim:
            claimName: test-pvc-2
```

You can create the resource with:

bash

```
$ kubectl apply -f deployment-mysql2.yaml
```

Create a `service2.yaml` with the following content:

service2.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mysql2
spec:
  ports:
  - port: 3306
    name: mysql
    targetPort: 3306
    nodePort: 32000
  selector:
```

```
    name: mysql2
  type: NodePort
```

You can create the resource with:

bash

```
$ kubectl apply -f service2.yaml
```

Testing the Deployment

You should be able to connect to MySQL using your favourite admin interface with the following credentials:

Credentials

```
Host: "replace with minikube ip"
Username: root
Password: password
Database: sample
Port: 32000
```

Create a table with:

SQL

```
CREATE TABLE colour (name VARCHAR(20));
```

Even if you delete the MySQL Pod, you should still be able to retrieve the same table.

You can list the Pods with:

bash

```
$ kubectl get pods
```

bash

```
$ kubectl delete pod <replace with Pod name>
```

The Pod was rescheduled, and the changes to the database are persisted.

Chapter 5

Lab

Extracting configs

The name of the database is stored in a Secret. But it isn't a secret as such, it's configuration.

You should store the value in a ConfigMap and refactor the Deployment to use that ConfigMap.